



Proyecto Final de Grado

Ciclo Formativo de Grado Superior
Desarrollo de Aplicaciones Multiplataforma

Curso Académico 2025/2026

IA-Lovers

Autor/es

Mikael Bryyovskyy Chirkova

Tutor/es

José Manuel Medina Ruiz

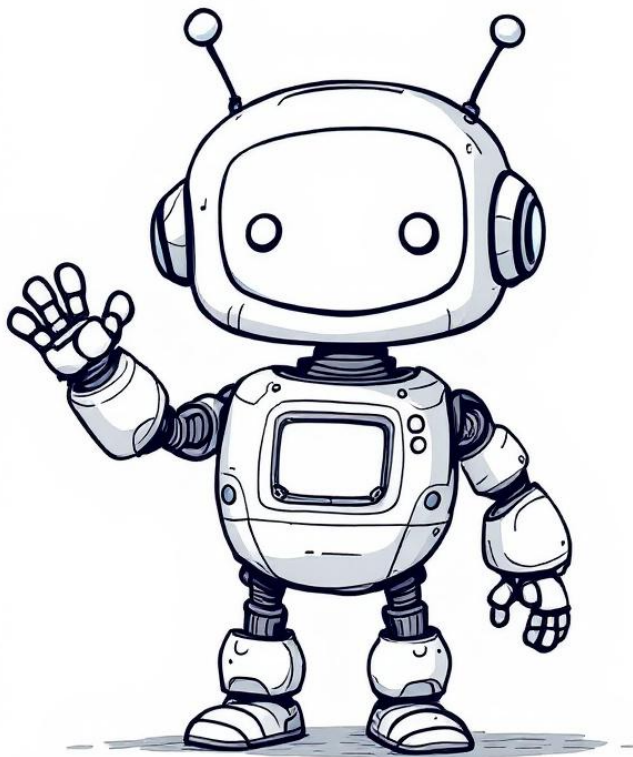
Departamento de Informática y Comunicaciones



WEB: <https://ia-lovers.vercel.app/>

APK: <https://github.com/mikaelbry/IA-Lovers/releases/tag/apk>

Código completo: <https://github.com/mikaelbry/IA-Lovers>





Índice de Contenidos

1. Introducción.....	7
1.1 Contexto y Motivación.....	7
1.1.1 Impacto en usuarios de internet y redes sociales.....	8
1.2 Propósito del proyecto.....	9
1.3 Importancia y necesidad del proyecto.....	9
1.4 Justificación de la solución propuesta.....	10
1.5 Objetivos del proyecto.....	10
2. Planificación del proyecto.....	12
2.1 Alcance del proyecto.....	12
2.2 Cronograma de trabajo.....	12
2.3 Metodología de desarrollo.....	14
2.4 Gestión de riesgos.....	15
3. Estado del arte.....	16
3.1 Investigación previa.....	16
3.2 Análisis de soluciones existentes.....	16
4. Requisitos.....	18
4.1 Análisis de requisitos.....	18
4.2 Especificación de casos de uso.....	25
4.3 Matriz de trazabilidad de requisitos.....	56
5. Diseño.....	62



5.1 Arquitectura del sistema.....	62
5.1.1 Diseño de la aplicación web.....	64
5.1.2 Diseño de la aplicación Android.....	66
5.1.3 Diseño del backend.....	69
5.1.4 Diseño de la base de datos.....	71
5.1.5 Diseño del almacenamiento de imágenes en Supabase Storage.....	75
5.2 Interacción y Flujo de la Aplicación.....	78
5.3 Diseño de la interfaz gráfica.....	80
6. Herramientas utilizadas.....	81
6.1 Herramientas Hardware.....	81
6.2 Herramientas Software.....	81
7. Implementación.....	83
7.1 Estrategia de desarrollo.....	83
7.2 Codificación y Desarrollo de Funcionalidades.....	84
7.3 Integración de los componentes.....	86
8. Pruebas.....	87
8.1 Pruebas en la etapa de desarrollo.....	87
8.2 Integración de los componentes.....	89
8.3 Pruebas de usuario, usabilidad y funcionalidad.....	90
8.4 Pruebas de Desempeño y de carga.....	91
8.5 Pruebas de Despliegue.....	92



9. Manuales.....	93
9.1 Manual de usuario.....	93
9.2 Manual de instalación.....	94
10. Posibles trabajos futuros.....	95
10.1 Mejoras propuestas.....	95
10.2 Expansión a nuevas plataformas.....	95
10.3 Integración con nuevas tecnologías.....	95
11. Conclusiones.....	96
11.1 Evaluación del proyecto.....	96
11.2 Lecciones aprendidas.....	96
12. Opinión personal.....	97
13. Glosario.....	98
14. Bibliografía.....	106

1. INTRODUCCIÓN

1.1 Contexto y Motivación

Actualmente vivimos en un momento en el que el contenido generado con Inteligencia Artificial ha irrumpido hasta el fondo de las redes sociales y el internet en general. Y parece que no ha hecho más que empezar.

Desde el lanzamiento de modelos avanzados como ChatGPT en 2022, la capacidad de generar texto, imágenes y videos mediante IA ha experimentado un crecimiento sin precedentes, impactando tanto la producción de contenidos como la experiencia de los usuarios en internet y redes sociales. Según un estudio de *Five Percent (2025)* ¹, en ese momento solo el 7,75% de los artículos web era generado por IA, pero un año después, la cifra ascendió a un 38,99%. En 2025, el contenido generado por IA superó la barrera del 50%, alcanzando un equilibrio con el contenido creado por humanos. Se puede ver claramente en esta gráfica:

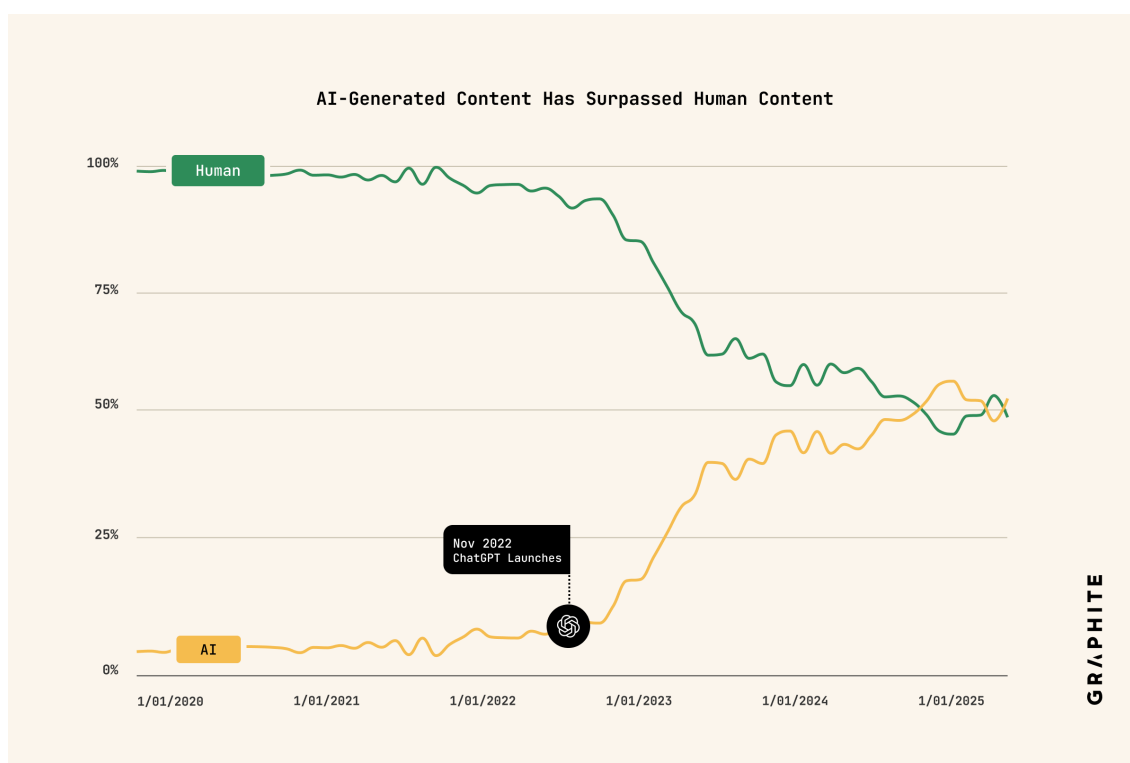


Figura 1. Aumento de artículos creados con IA



1.1.1 IMPACTO EN USUARIOS DE INTERNET Y REDES SOCIALES

Efectos positivos

La IA ha mejorado la experiencia de los usuarios en redes sociales mediante la personalización de contenidos, lo que incrementa la retención. La generación automática de texto alternativo para imágenes y subtítulos para videos ha mejorado la accesibilidad para usuarios con discapacidades. Además, la IA contribuye a la moderación de contenidos, filtrando publicaciones inapropiadas y mejorando la seguridad en las plataformas. La creación de contenido hiperpersonalizado en tiempo real y la optimización algorítmica han permitido a las marcas conectar mejor con sus audiencias, aumentando la relevancia y el impacto de las publicaciones .

Efectos negativos

La proliferación de contenido generado por IA ha generado una avalancha de materiales de baja calidad (“AI slop”), que afectan negativamente la experiencia de usuario y la percepción de autenticidad en redes sociales. La saturación de contenidos artificiales ha llevado a una pérdida de autenticidad, con usuarios que prefieren contenidos humanos con imperfecciones visibles, asociando la perfección con artificialidad. La desinformación y la dificultad para distinguir contenido real de generado por IA han erosionado la confianza en las redes sociales como fuentes de información fiable. Esto incrementa la fatiga cognitiva digital y la desconfianza hacia el contenido generado por IA

Percepción y preferencias de los usuarios

El 65,4% de los usuarios percibe contenido generado por IA con frecuencia, pero solo el 24,9% está seguro de identificarlo (*MarketingNews. 2025, 26 de junio*)². La mayoría prefiere contenido auténtico y humano, mostrando escepticismo hacia el contenido generado por IA(*Congreso Nodos. 2024*)³.



1.2 PROPÓSITO DEL PROYECTO

El propósito de este proyecto es el desarrollo de una red social multiplataforma (para Android y para web), llamada IA-Lovers, orientada exclusivamente a la publicación y consumo de contenido generado mediante inteligencia artificial. La plataforma permitirá a los usuarios compartir, visualizar e interactuar con imágenes producidas por IA.

1.3 IMPORTANCIA Y NECESIDAD DEL PROYECTO

Al crear un espacio digital exclusivamente para el contenido hecho con IA, se busca beneficiar a ambos perfiles de usuarios en internet, tanto los que no desean ver este tipo de contenido como los que sí. Ya que con esta plataforma se busca redirigir a todos los creadores posibles a esta red, de tal manera que su público siempre estará interesado en el contenido hecho con IA. Mientras que por otra parte también podría contribuir a reducir la saturación de contenido generado por IA en redes sociales existentes, que será del agrado de la mayoría de usuarios.

1) Problemas que se buscan resolver:

- a) Dificultad para encontrar contenido generado por IA de forma organizada.
- b) Confusión entre contenido humano y contenido artificial.
- c) Fricción entre usuarios con intereses opuestos dentro de las mismas redes sociales.

2) Oportunidades que se desean aprovechar:

- a) Centralizar este tipo de contenido en un único entorno.
- b) Ofrecer una experiencia adaptada a los intereses del público objetivo.
- c) Fomentar una comunidad centrada en la exploración, experimentación y análisis de la inteligencia artificial como herramienta creativa.



1.4 JUSTIFICACIÓN DE LA SOLUCIÓN PROPUESTA

La solución propuesta resulta pertinente debido al crecimiento continuo del contenido generado mediante inteligencia artificial y a la ausencia de plataformas especializadas centradas exclusivamente en este tipo de publicaciones. Actualmente, las redes sociales generalistas integran contenido creado tanto por humanos como por sistemas de IA dentro del mismo entorno, lo que puede generar problemas relacionados con la saturación de contenido artificial, la dificultad de identificación y la fragmentación de comunidades con intereses específicos.

En este contexto, el desarrollo de una plataforma dedicada exclusivamente al contenido generado por inteligencia artificial permite ofrecer un entorno más organizado y adaptado a las necesidades de este público objetivo. La especialización de la red social facilitaría la clasificación y descubrimiento de contenido generado por IA, además de fomentar la interacción entre usuarios interesados en herramientas creativas basadas en inteligencia artificial.



1.5 OBJETIVOS DEL PROYECTO

El objetivo principal de este proyecto es desarrollar una red social multiplataforma, accesible desde web y dispositivos Android, orientada a la publicación y consumo de contenido generado mediante inteligencia artificial. La plataforma deberá garantizar la sincronización de la información entre ambas interfaces cliente, utilizando un backend y una base de datos comunes que permitan la gestión centralizada del sistema.

Objetivos específicos:

1. Implementar las funcionalidades básicas de una red social, incluyendo publicación de contenido, sistema de “me gusta”, comentarios, perfiles de usuario, seguimiento entre usuarios y gestión de cuentas.
2. Diseñar una arquitectura cliente-servidor compartida entre la aplicación web y la aplicación Android, asegurando la interoperabilidad y consistencia de los datos en tiempo real.
3. Implementar mecanismos de autenticación y seguridad que garanticen la protección de la información de los usuarios, incluyendo autenticación mediante correo electrónico y contraseña, validación de entradas, protección frente a inyecciones SQL, uso de tokens de autenticación expirables y comunicación segura mediante HTTPS.
4. Incorporar medidas de protección adicionales frente a amenazas comunes en aplicaciones web y móviles, como configuraciones seguras de CORS y sistemas CAPTCHA anti-bots.
5. Diseñar un sistema de almacenamiento eficiente y seguro en la base de datos, aplicando algoritmos robustos de hash para el almacenamiento de contraseñas y controles estrictos sobre los formatos de archivos permitidos en las subidas de contenido.



2. PLANIFICACIÓN DEL PROYECTO

2.1 Alcance del proyecto

El alcance del proyecto se centrará en la construcción de un prototipo funcional que permita demostrar la viabilidad técnica de la plataforma y la integración entre las distintas tecnologías utilizadas. Se priorizará la implementación de las funcionalidades esenciales de interacción social y gestión de usuarios, así como la seguridad básica de la aplicación y la sincronización de datos entre plataformas.

Debido a las limitaciones de tiempo y recursos propias de un Trabajo de Fin de Grado, determinadas características avanzadas quedarán fuera del alcance del proyecto. Entre ellas se encuentran la generación integrada de contenido mediante modelos IA, la mensajería en tiempo real, la moderación automática avanzada y la adaptación a plataformas distintas de Android y web.

Asimismo, el proyecto no contempla la implementación de sistemas comerciales de monetización o publicidad, centrándose principalmente en el desarrollo funcional y técnico de la plataforma.

2.2 CRONOGRAMA DE TRABAJO

El desarrollo del proyecto se ha organizado en cuatro fases principales, con el objetivo de estructurar el trabajo de forma progresiva y asegurar una integración correcta entre backend, aplicación web, aplicación Android y despliegue final.

Primera fase: desarrollo de la base del sistema.

Durante esta etapa se implementa el backend común para ambas interfaces cliente y se diseña la estructura inicial de la base de datos. También se definen los modelos principales de la aplicación, la lógica de acceso a datos y la comunicación entre el servidor y los clientes. Esta fase resulta fundamental porque establece la arquitectura base sobre la que se construirá el resto del proyecto.



Segunda fase: desarrollo de la aplicación web.

Una vez consolidada la base del sistema, se desarrolla la versión web de la plataforma con mayor profundidad. En esta etapa se implementan las principales vistas, componentes de interacción y funcionalidades orientadas al uso desde navegador. Además, se trabaja en la identidad visual del proyecto, buscando un diseño coherente con la temática de IA-Lovers y con una apariencia diferenciada que refuerce la marca de la plataforma.

Tercera fase: desarrollo de la aplicación Android.

En esta fase se construye la aplicación móvil para Android, conectándola al mismo backend utilizado por la web. El objetivo es garantizar la sincronización de datos entre plataformas y mantener una experiencia de uso consistente. También se adapta la interfaz al formato móvil, procurando conservar una línea visual similar a la de la versión web, pero ajustada a las necesidades de navegación en dispositivos Android.

Cuarta fase: pruebas, ajustes y despliegue.

En la última etapa se realizan tareas de revisión, corrección de errores y pulido general del sistema. Asimismo, se conectan la base de datos, el backend y la web al entorno de producción, dejando preparada la plataforma para su uso final. Esta fase incluye también la verificación del funcionamiento general de la aplicación, tanto en web como en Android, con el fin de asegurar la estabilidad y la coherencia del conjunto.

Duración de cada fase:

- 1) Primera fase: 1 mes, desde mediados de enero de 2026 hasta mediados de febrero.
- 2) Segunda fase: 2 meses, desde mediados de febrero hasta mediados de abril.
- 3) Tercera fase: 1 mes, desde mediados de abril hasta mediados de mayo.
- 4) Cuarta fase: 2 semanas, durante las dos últimas semanas de mayo.



2.3 Metodología de desarrollo

Para el desarrollo del proyecto se ha utilizado una metodología ágil basada en Kanban. Este enfoque permite organizar el trabajo de forma visual mediante tareas divididas en diferentes estados, facilitando el seguimiento del progreso y la gestión de prioridades durante el desarrollo.

La metodología Kanban ha resultado adecuada para este proyecto debido a que el desarrollo se ha realizado de forma progresiva e incremental, implementando primero las funcionalidades principales y posteriormente añadiendo mejoras, correcciones y nuevas características. Además, este enfoque ofrece una mayor flexibilidad ante posibles cambios o ajustes surgidos durante el proceso de desarrollo.

Las tareas del proyecto se han dividido en diferentes bloques, correspondientes al backend, la aplicación web, la aplicación Android y el despliegue final. Esto ha permitido mantener una organización clara del trabajo y controlar el avance de cada parte del sistema de manera independiente.

El uso de una metodología ágil también ha favorecido la detección temprana de errores y la realización de pruebas continuas durante el desarrollo, permitiendo mejorar progresivamente la estabilidad y funcionalidad de la plataforma.



2.4 Gestión de riesgos

Durante el desarrollo del proyecto se identificaron distintos riesgos técnicos y organizativos que podían afectar al progreso y a la finalización del sistema dentro del tiempo previsto.

Uno de los principales riesgos fue la falta de experiencia previa en el desarrollo de aplicaciones móviles para Android. Aunque contaba con conocimientos de programación y desarrollo web, el entorno móvil suponía una tecnología nueva, lo que implicaba una curva de aprendizaje adicional y la posibilidad de encontrar dificultades técnicas durante la implementación de la aplicación.

Otro riesgo importante estuvo relacionado con la disponibilidad de tiempo. El desarrollo del proyecto tuvo que compaginarse con la carga académica diaria, tanto en horario de mañana como de tarde, reduciendo significativamente las horas disponibles para trabajar en el TFG.

Además, también existía el riesgo de problemas de integración entre la aplicación web, la aplicación Android y el backend compartido. Para reducir esta posibilidad, se diseñó una arquitectura centralizada basada en una API común y se realizaron pruebas frecuentes de comunicación entre los distintos componentes del sistema.



3. ESTADO DEL ARTE

3.1 Investigación previa

La expansión de la IA generativa está transformando el ecosistema informativo y plantea desafíos significativos. Organizaciones como la UNESCO advierten que aplicaciones cada vez más sofisticadas de IA (textos, imágenes, videos) dificultan la verificación de la información, ampliando los riesgos de desinformación y deepfakes(UNESCO. 2026, 11 de marzo)⁴.

En el ámbito profesional, encuestas recientes revelan que los gestores de redes sociales ya integran ampliamente la IA: por ejemplo, el 66% de ellos afirma usarla para al menos la mitad del contenido que publican (Bustos, D. 2025, 19 de agosto)⁵. Estos datos sugieren que pronto veremos una gran proporción de contenido online generado por IA, con implicaciones tanto en la calidad de la información como en las preferencias y expectativas de los usuarios.

3.2 Análisis de soluciones existentes

Hasta la fecha no existe una red social consolidada enfocada exclusivamente en arte generativo por IA, pero sí han surgido proyectos y aplicaciones relacionados, cada uno con sus particularidades:

Moltbook: plataforma lanzada en 2026 que funciona como un “Reddit de IA”, donde solo interactúan agentes de IA (bots). No hay usuarios humanos participantes; los agentes crean debates, historias e incluso religiones propias. Aunque es un experimento interesante, Moltbook está dirigido a la interacción entre programas inteligentes y no facilita la publicación de obras visuales de IA por usuarios reales. IA-Lovers se diferenciaría en este aspecto: está dirigida a usuarios humanos que quieren compartir y disfrutar arte generado por IA, no al diálogo entre máquinas

Fomo – AI Social Network: aplicación móvil (Android/iOS) que presenta una experiencia social “inmersiva” basada enteramente en personajes de IA. En Fomo no hay usuarios reales: todas las cuentas, comentarios y “me gusta” son creados por agentes de



IA. El usuario publica fotos y recibe instantáneamente reacciones generadas artificialmente (comentarios, likes, hasta «fans» que le siguen), como se muestra en la figura siguiente. Fomo promete un entorno sin “toxicidad ni presión” social, pero su enfoque es distinto: simula una red social completa con personajes automatizados. En cambio, IA-Lovers busca la interacción genuina entre personas creativas, usando la IA solo como herramienta productiva.

Plataformas de vídeo generativo (Meta Sora/Vibes): las grandes tecnológicas están apostando por redes dedicadas a contenidos de IA. OpenAI prepara Sora 2, una red social de vídeos verticales similar a TikTok donde solo se podrán compartir clips generados por IA. Meta lanzó Vibes, un feed dentro de su app Meta AI donde los usuarios crean y consumen vídeos breves hechos con IA. Estas plataformas ofrecen herramientas de edición y un feed personalizado, pero se centran en vídeo. IA-Lovers difiere en que está orientada principalmente a imágenes fijas generadas por IA, ofreciendo un espacio para compartir ilustraciones y arte digital, más que vídeos.

En conjunto, las soluciones existentes ilustran enfoques diversos. Algunas plataformas recientes (Moltbook, Fomo, Sora/Vibes) exploran comunidades centradas en la IA (puramente automatizadas o híbridas). IA-Lovers propone una alternativa única: una red social orientada a usuarios reales que desean publicar y disfrutar exclusivamente contenido visual generado por IA.



4. REQUISITOS

4.1 Análisis de requisitos

A continuación se presentan los requisitos funcionales y no funcionales identificados para el desarrollo de la plataforma IA-Lovers.

Requisitos funcionales:

Tabla 1: Requisitos funcionales del módulo de autenticación y gestión de cuentas

ID	Descripción
RF-01	El sistema permitirá el registro de nuevos usuarios mediante correo electrónico, nombre de usuario y contraseña, con verificación de un captcha anti-bots y por código de 6 dígitos enviado por correo.
RF-02	El sistema permitirá el inicio de sesión mediante correo electrónico y contraseña, protegiendo el acceso con un captcha anti-bots.
RF-03	El sistema permitirá la recuperación de contraseña mediante un flujo de 3 pasos: solicitud con captcha, verificación por código de 6 dígitos y establecimiento de nueva contraseña.
RF-04	El sistema permitirá al usuario cerrar sesión, invalidando el token de autenticación actual.
RF-05	El sistema permitirá al usuario actualizar su nombre de usuario, correo electrónico (con verificación previa) y contraseña, requiriendo la contraseña actual como confirmación.
RF-06	El sistema permitirá al usuario cambiar su foto de avatar mediante subida de archivo.
RF-07	El sistema permitirá al usuario eliminar su cuenta de forma permanente, requiriendo confirmación por contraseña y texto explícito.



Tabla 2: Requisitos funcionales del módulo de publicaciones

ID	Descripción
RF-08	El sistema permitirá al usuario autenticado crear publicaciones con una imagen, un título, una descripción opcional y etiquetas opcionales.
RF-09	El sistema permitirá al usuario autenticado eliminar sus propias publicaciones.
RF-10	El sistema permitirá a cualquier usuario visualizar el feed general de publicaciones con paginación por cursor infinito.
RF-11	El sistema permitirá a cualquier usuario visualizar el detalle de una publicación individual con su información completa (el número de "me gusta" y todos los comentarios).
RF-12	El sistema permitirá al usuario autenticado dar y quitar "me gusta" a las publicaciones.
RF-13	El sistema permitirá a cualquier usuario explorar publicaciones con filtros por búsqueda textual y ordenación (más recientes, más "me gusta").
RF-14	El sistema permitirá al usuario autenticado visualizar un feed con las publicaciones de los usuarios que sigue.

Tabla 3: Requisitos funcionales del módulo de comentarios

ID	Descripción
RF-15	El sistema permitirá al usuario autenticado comentar en cualquier publicación.
RF-16	El sistema permitirá al usuario autenticado eliminar sus propios comentarios.
RF-17	El sistema permitirá comentarios anidados (respuestas a comentarios existentes).

Tabla 4: Requisitos funcionales del módulo de interacción social

ID	Descripción
RF-18	El sistema permitirá al usuario autenticado seguir y dejar de seguir a otros usuarios.
RF-19	El sistema permitirá a cualquier usuario visualizar el perfil público de otro usuario con sus publicaciones, número de seguidores y seguidos.
RF-20	El sistema permitirá al usuario autenticado visualizar sus propios seguidores y seguidos mediante ventanas modales.

Tabla 5: Requisitos funcionales del módulo de notificaciones

ID	Descripción
RF-21	El sistema notificará al usuario cuando alguien le siga, le dé "me gusta" a una publicación suya, comente en una publicación suya o responda a un comentario suyo.
RF-22	El sistema permitirá al usuario autenticado visualizar la lista de notificaciones con indicador de no leídas.
RF-23	El sistema permitirá al usuario marcar todas las notificaciones como leídas.
RF-24	El sistema mostrará un contador de notificaciones no leídas en la interfaz.

Tabla 6: Requisitos funcionales del módulo de etiquetas

ID	Descripción
RF-25	El sistema permitirá buscar etiquetas existentes.
RF-26	El sistema permitirá al usuario autenticado crear nuevas etiquetas.

Tabla 7: Requisitos funcionales del módulo multiplataforma



ID	Descripción
RF-27	El sistema proporcionará una API REST común que garantice la sincronización de datos entre la aplicación web y la aplicación Android.
RF-28	La aplicación Android implementará las mismas funcionalidades que la versión web, adaptadas al formato móvil nativo.

Requisitos no funcionales:

Tabla 8: Requisitos no funcionales agrupados por categorías

ID	Descripción	Categoría
RNF-01	Las contraseñas se almacenarán utilizando bcrypt con un cost factor de 12, garantizando la protección frente a ataques de fuerza bruta.	Seguridad
RNF-02	La autenticación se realizará mediante tokens opacos de 64 caracteres hexadecimales (256 bits de entropía) generados con <i>random_bytes(32)</i> , con expiración a los 90 días y renovación automática en cada uso.	Seguridad
RNF-03	Las comunicaciones con la base de datos utilizarán exclusivamente sentencias preparadas (PDO prepared statements) para prevenir inyecciones SQL.	Seguridad
RNF-04	La subida de archivos estará restringida a los formatos JPEG, PNG y WebP, con un tamaño máximo de 4 MB, validando el tipo MIME real mediante <i>finfo</i> .	Seguridad
RNF-05	Los endpoints de autenticación incorporarán protección Altcha (proof-of-work) para prevenir ataques automatizados.	Seguridad
RNF-06	Los endpoints de autenticación dispondrán de limitación de tasa (rate limiting) basada en archivo, con ventanas deslizantes configurables (p.ej., 8 intentos en 300 segundos para login).	Seguridad



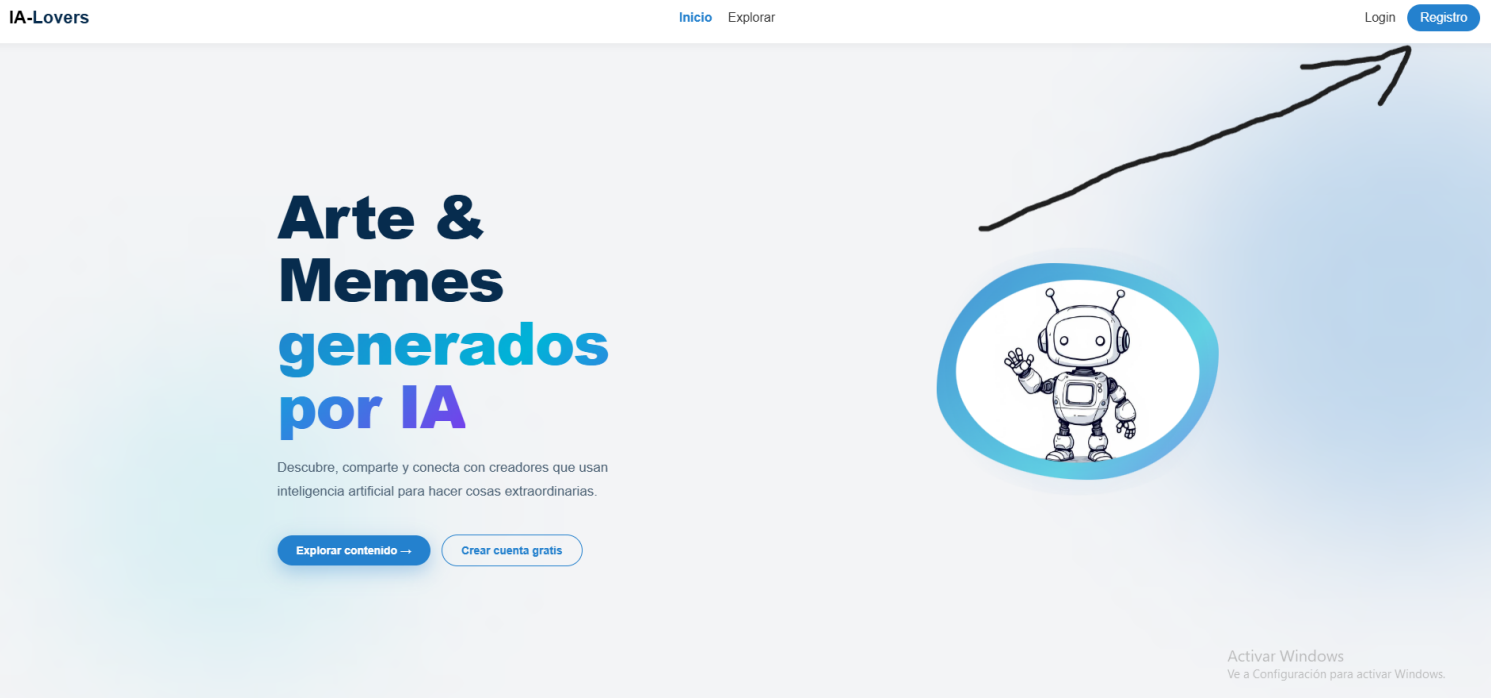
ID	Descripción	Categoría
RNF-07	El API incluirá cabeceras de seguridad: <i>X-Frame-Options: DENY</i> , <i>X-Content-Type-Options: nosniff</i> , <i>X-XSS-Protection</i> , <i>Referrer-Policy: no-referrer</i> y <i>Cache-Control: no-store</i> .	Seguridad
RNF-08	La aplicación web será responsive, adaptándose a distintos tamaños de pantalla (escritorio, tablet y móvil).	Usabilidad
RNF-9	Las interfaces de usuario tanto web como Android mantendrán una línea visual coherente, con la misma paleta de colores, tipografía y estilo de componentes.	Usabilidad
RNF-10	El sistema utilizará un sistema de almacenamiento externo (Supabase Storage) para imágenes de publicaciones y avatares, con URLs públicas.	Arquitectura
RNF-11	La aplicación web será desplegable en Vercel con runtime PHP 8.3.	Despliegue
RNF-12	La aplicación Android utilizará Retrofit para la comunicación con el API, garantizando un manejo eficiente de las peticiones HTTP.	Tecnología
RNF-13	El feed de publicaciones implementará scroll infinito con paginación basada en cursor para un rendimiento óptimo.	Rendimiento

4.2 Especificación de casos de uso

A continuación se detallarán algunos casos de uso. Para algunos se mostrarán capturas de la interfaz web y para otros los de la interfaz de la app, pero suelen ser prácticamente idénticos en ambos casos.

Caso de uso 1: Registro de usuario (web)

- Actores: Usuario no registrado
- Flujo principal:
 1. El usuario accede a la pantalla de registro





2. El sistema muestra el formulario con campos de nombre de usuario, correo electrónico, contraseña y confirmación de contraseña, además del captcha

Crear cuenta

Únete a la comunidad IA

☐ I'm not a robot

Protected by ALTCHA

Continuar

¿Ya tienes cuenta? [Iniciar sesión](#)

Activar Windows
Ve a Configuración para activar Windows.



3. El usuario completa los campos y resuelve el captcha

Crear cuenta

Unete a la comunidad IA

☒ Verified

Protected by ALTCHA

Verificación completada. Ya puedes continuar.

Continuar

¿Ya tienes cuenta? [Iniciar sesión](#)



4. El sistema valida los datos y, si son correctos, envía un código de verificación de 6 dígitos al correo electrónico

Tu código de verificación de IA-Lovers Inbox x



IA-Lovers <ialoversenterprise@gmail.com>
to usuarioDeCasosDeUso ▾

7:45 PM (0 minutes ago)

Verifica tu correo

Hola usuarioDeCasosDeUso,

Usa este código para terminar tu registro en IA-Lovers:

2 5 6 9 6 4

El código caduca en 10 minutos.

Si tu no intentaste crear esta cuenta, puedes ignorar este correo.

5. El usuario introduce el código recibido

Verificar correo

Introduce el código de 6 dígitos enviado a
us*****@gmail.com.

256964

Te hemos enviado un código al correo indicado.

Crear cuenta

[Reenviar código](#) [Cancelar registro](#)

¿Ya tienes cuenta? [Iniciar sesión](#)

6. El sistema verifica el código y crea la cuenta

Bienvenido

Inicia sesión para continuar



☐ I'm not a robot

Protected by ALTCHA

Cuenta creada correctamente. Completa el CAPTCHA e inicia sesión.

Entrar

[He olvidado mi contraseña](#)

[¿No tienes cuenta? Crear cuenta](#)

Postcondiciones: El usuario queda registrado y puede iniciar sesión



Flujos alternativos:

4a. Si el correo o nombre de usuario ya existen, el sistema muestra un mensaje de error

Crear cuenta

Unete a la comunidad IA





☐ I'm not a robot 

Protected by ALTCHA

Email ya registrado

Continuar

¿Ya tienes cuenta? [Iniciar sesion](#)



5a. Si el código es incorrecto, el sistema incrementa el contador de intentos; tras 5 fallos, invalida el registro pendiente

Verificar correo

Introduce el código de 6 dígitos enviado a
ju*****@gmail.com.

Código incorrecto demasiadas veces. El registro pendiente ha sido cancelado

[Crear cuenta](#)

[Reenviar código](#) [Cancelar registro](#)

¿Ya tienes cuenta? [Iniciar sesión](#)

5b. Si el código caduca (10 minutos), el sistema no permite el registro

Verificar correo

Introduce el código de 6 dígitos enviado a
ju*****@gmail.com.

El registro pendiente no existe o ya ha caducado

[Crear cuenta](#)

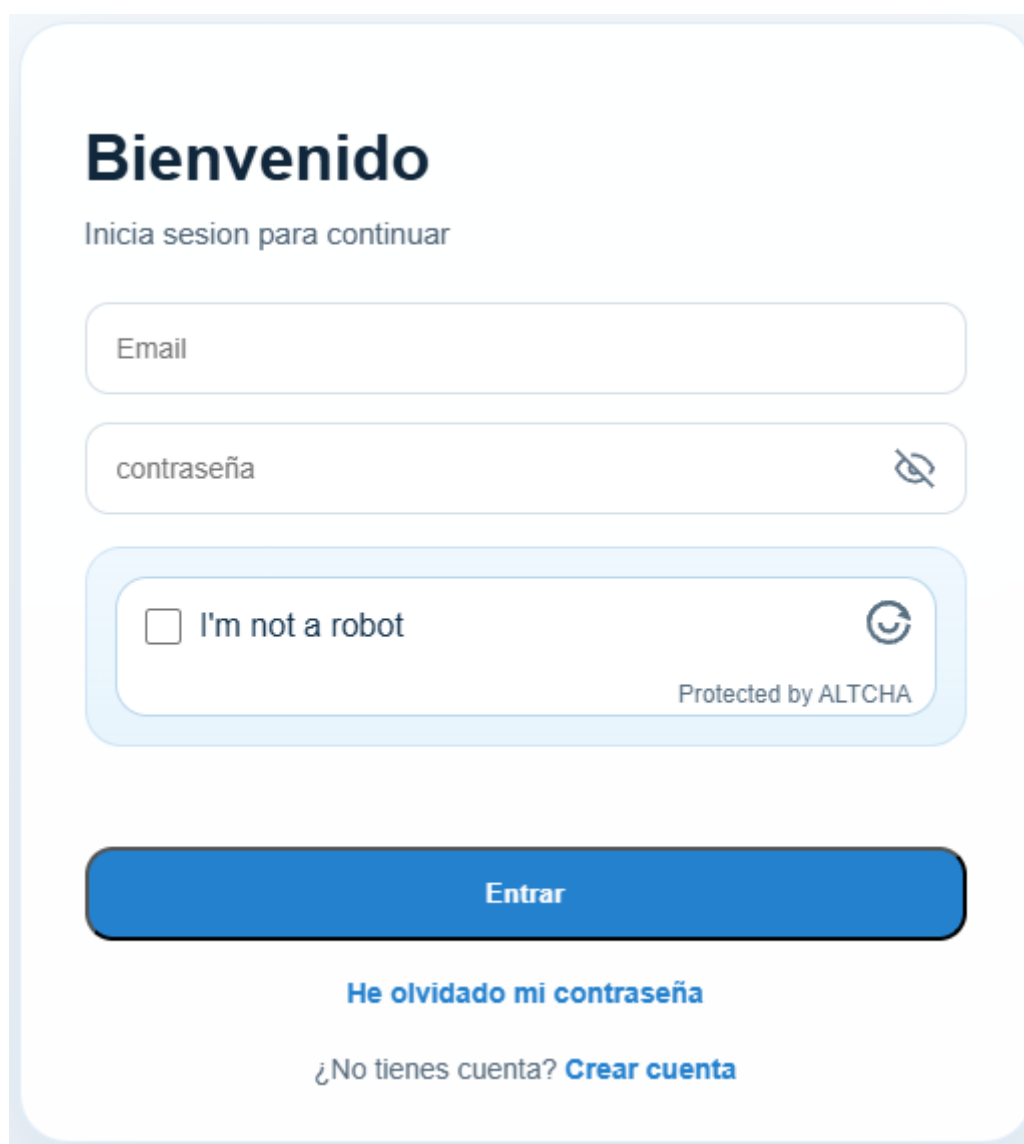
[Reenviar código](#) [Cancelar registro](#)

¿Ya tienes cuenta? [Iniciar sesión](#)



CASO DE USO 2: INICIO DE SESIÓN (WEB)

- Actores: Usuario registrado
- Precondiciones: El usuario tiene una cuenta activa
- Flujo principal:
 1. El usuario accede a la pantalla de inicio de sesión y el sistema muestra el formulario con campos de correo electrónico, contraseña y captcha anti-bots



The image shows a login form with the following elements:

- Header:** "Bienvenido" (Welcome) in large bold text, followed by "Inicia sesion para continuar" (Log in to continue).
- Email Field:** A text input field labeled "Email".
- Password Field:** A text input field labeled "contraseña" (password) with a toggle icon for visibility.
- Captcha:** A checkbox labeled "I'm not a robot" with a circular refresh icon to its right.
- Security:** The text "Protected by ALTCHA" is displayed below the captcha.
- Login Button:** A large blue button labeled "Entrar" (Log in).
- Links:** Below the button, there are two links: "He olvidado mi contraseña" (I forgot my password) and "¿No tienes cuenta? [Crear cuenta](#)" (Don't you have an account? Create account).

2. El usuario introduce sus credenciales y resuelve el captcha

Bienvenido

Inicia sesión para continuar

.....



☒ Verified



Protected by ALTCHA

Verificación completada. Ya puedes continuar.

Entrar

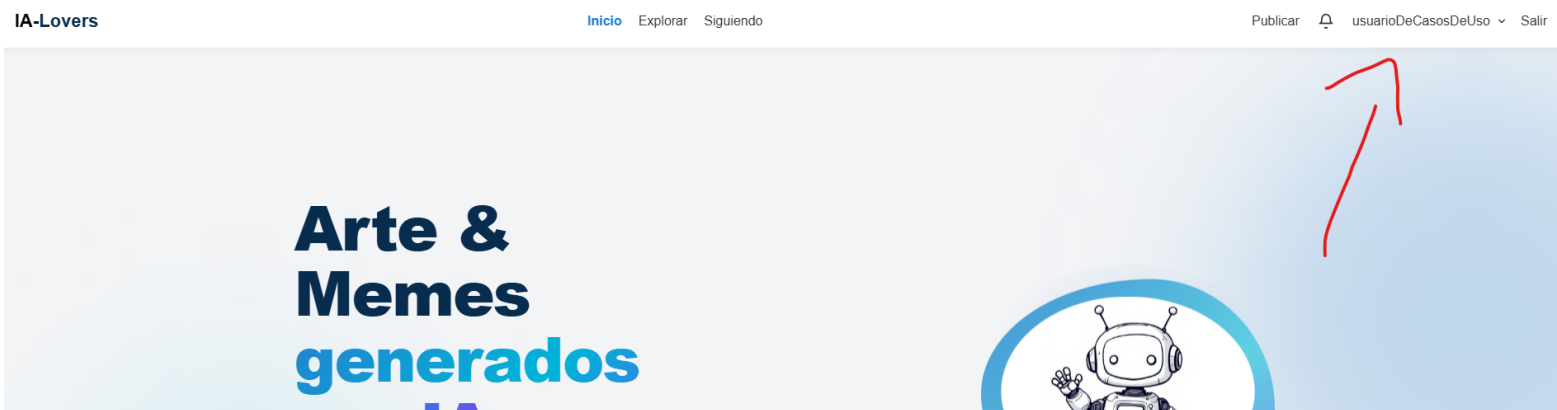
[He olvidado mi contraseña](#)

¿No tienes cuenta? [Crear cuenta](#)

3. El sistema verifica las credenciales
4. El sistema genera un token de autenticación y lo almacena en el dispositivo



5. El sistema redirige al usuario a la pantalla principal



Postcondiciones: El usuario queda autenticado en la plataforma

Flujos alternativos:

4a. Si las credenciales son incorrectas, el sistema muestra un mensaje de error

Bienvenido

Inicia sesión para continuar

usuarioDeCasosDeUso@gmail.com

.....

☐ I'm not a robot

Protected by ALTCHA

Credenciales incorrectas

Entrar

[He olvidado mi contraseña](#)

¿No tienes cuenta? [Crear cuenta](#)



4b. Si se superan 8 intentos fallidos en 5 minutos, el sistema bloquea temporalmente nuevas peticiones

Bienvenido

Inicia sesión para continuar



☐ I'm not a robot



Protected by ALTCHA

Demasiadas solicitudes

Entrar

[He olvidado mi contraseña](#)

¿No tienes cuenta? [Crear cuenta](#)



Caso de uso 3: Publicar contenido (web)

- Actores: Usuario autenticado
- Precondiciones: El usuario ha iniciado sesión
- Flujo principal:
 1. El usuario accede a la pantalla de creación de publicación y el sistema muestra el formulario con selector de imagen, campo de título, descripción y etiquetas

The screenshot shows the 'Crear nuevo Post' (Create new Post) form in the IA-Lovers application. The form is centered on a light gray background. At the top, there's a navigation bar with 'Inicio', 'Explorar', and 'Siguiendo' on the left, and 'Publicar', a bell icon, 'usuarioDeCasosDeUso', and 'Salir' on the right. The form itself has a title 'Crear nuevo Post' and contains the following elements: a 'Título' (Title) input field, a 'Descripción' (Description) text area, a section for 'Agregar tags' (Add tags) with pre-defined tags like #Meme, #Arte, #ChatGPT, #Mistral AI, #MidJourney, and #Stable Diffusion, a link for 'Otros tags' (Other tags), a file selection button labeled 'Seleccionar archivo' with the text 'Ningún archivo seleccionado' (No file selected), and a blue 'Publicar' (Publish) button at the bottom. In the bottom right corner of the page, there is a small cartoon robot character and a Windows watermark that says 'Activar Windows. Ve a Configuración para activar Windows.'

2. El usuario selecciona una imagen (JPEG, PNG o WebP, máximo 4 MB), introduce el título (máximo 80 caracteres), la descripción opcional (máximo 500 caracteres) y las etiquetas opcionales (máximo 4)

Crear nuevo Post

Muchos ojos

Hecho con Seedream

Agregar tags

#Meme #Arte #ChatGPT #Mistral AI #MidJourney #Stable Diffusion

#ChatGPT x #Mistral AI x #MidJourney x #Stable Diffusion x

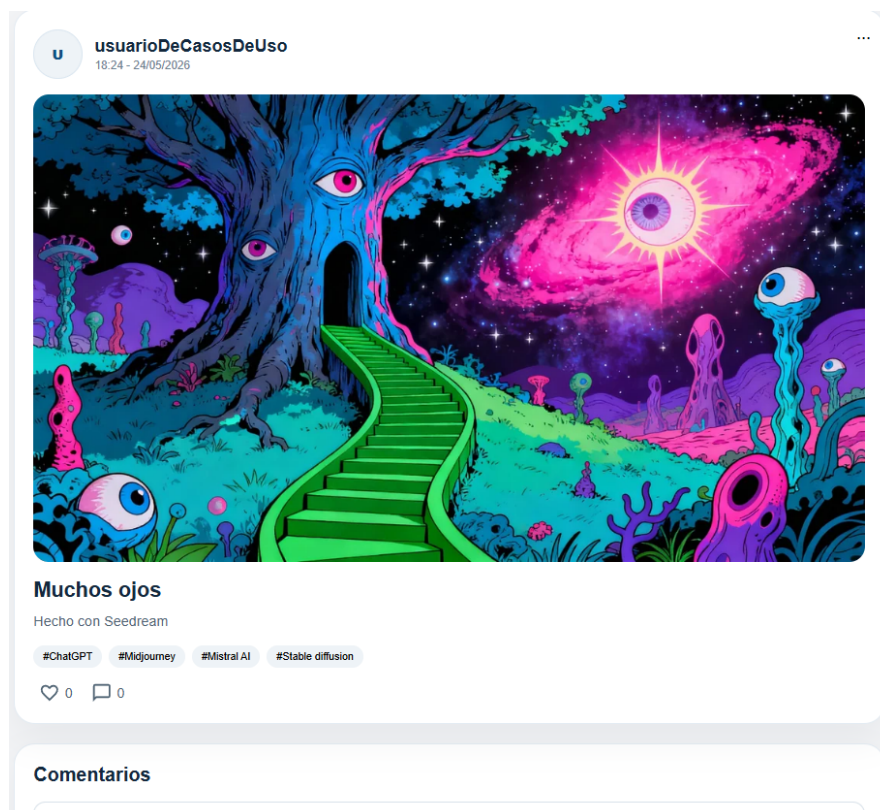
Alcanzado máximo número de tags

Otros tags ▼

Seleccionar archivo ojos.webp

Publicar

- El usuario envía el formulario y el sistema valida los datos, sube la imagen al almacenamiento, crea la publicación y redirige al usuario a esta





Postcondiciones: La publicación aparece en el feed y en el perfil del usuario

Flujos alternativos:

3a. Si la imagen no cumple los requisitos de formato o tamaño, el sistema muestra un mensaje de error

Ningún archivo seleccionado

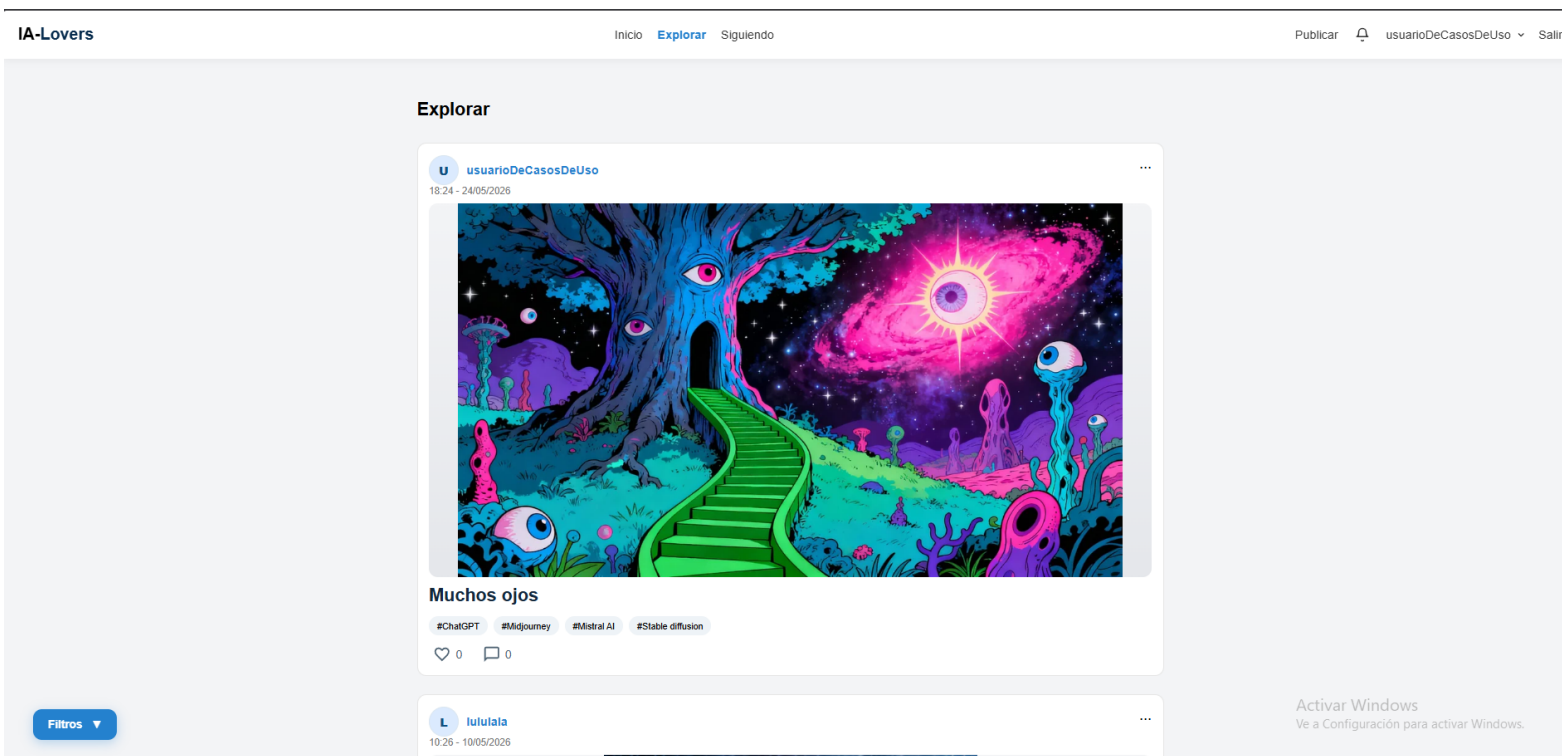
Formato no permitido (solo JPG, PNG, WEBP)

Ningún archivo seleccionado

Archivo demasiado grande (máx 4MB)

Caso de uso 4: Explorar publicaciones

- Actores: Cualquier usuario (autenticado o no)
- Precondiciones: Ninguna
- Flujo principal:
 1. El usuario accede a la pantalla de explorar y el sistema muestra el feed de publicaciones ordenadas por más recientes



2. El usuario puede desplazarse hacia abajo para cargar más publicaciones (scroll infinito)
3. El usuario puede aplicar filtros de búsqueda por texto y cambiar el orden de visualización

Más recientes

▼

Quitar filtros

Aplicar filtros

Filtros ▲

Postcondiciones: El usuario visualiza las publicaciones según los filtros seleccionados

Inicio [Explorar](#) Siguiendo

Explorar

 **jacksonn**
10:52 - 20/02/2026



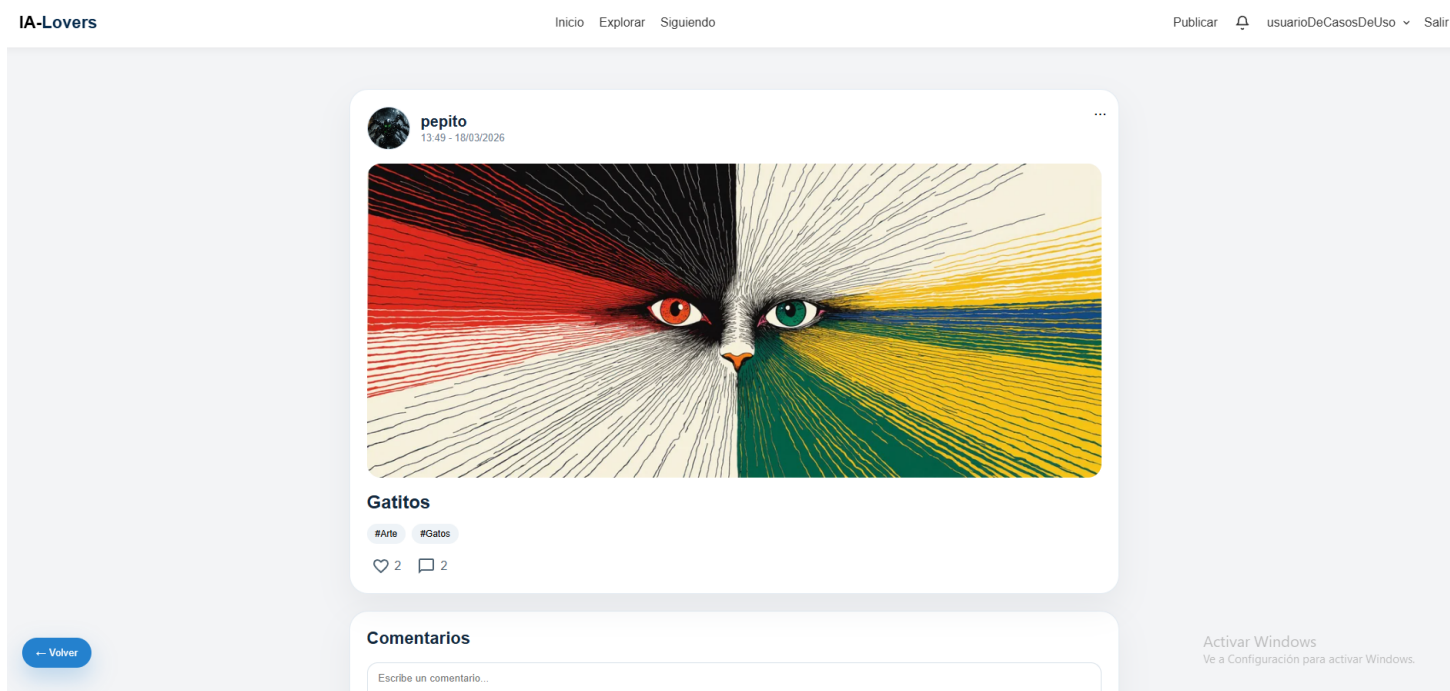
Messi comiendo paella encima de un tiburón

#Meme

♡ 10 💬 25

Caso de uso 5: Interactuar con una publicación (web)

- Actores: Usuario autenticado
- Precondiciones: El usuario ha iniciado sesión
- Flujo principal:
 1. El usuario visualiza una publicación en el feed o en el detalle



2. El usuario puede dar "me gusta" pulsando el icono correspondiente



3. El usuario puede escribir un comentario en el campo de texto y enviarlo

Comentarios

Escribe un comentario...

Comentar

**pepito**
19:07 - 05/04/2026

Bonito

Responder 1 respuestas

**usuarioDeCasosDeUso**
19:00 - 24/05/2026

Eliminar

Esto es un comentario para los casos de uso

Responder

4. El usuario puede responder a un comentario existente

Comentarios

← Volver al hilo anterior

**usuarioDeCasosDeUso**
19:00 - 24/05/2026

Eliminar

Esto es un comentario para los casos de uso

Responder 1 respuestas

Escribe un comentario...

Comentar

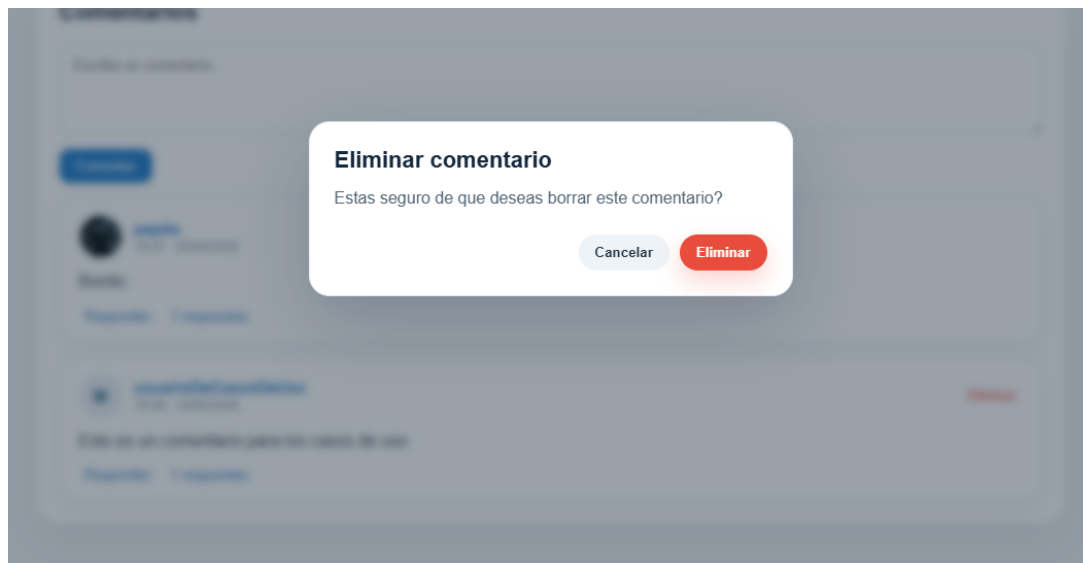
**usuarioDeCasosDeUso**
19:02 - 24/05/2026

Eliminar

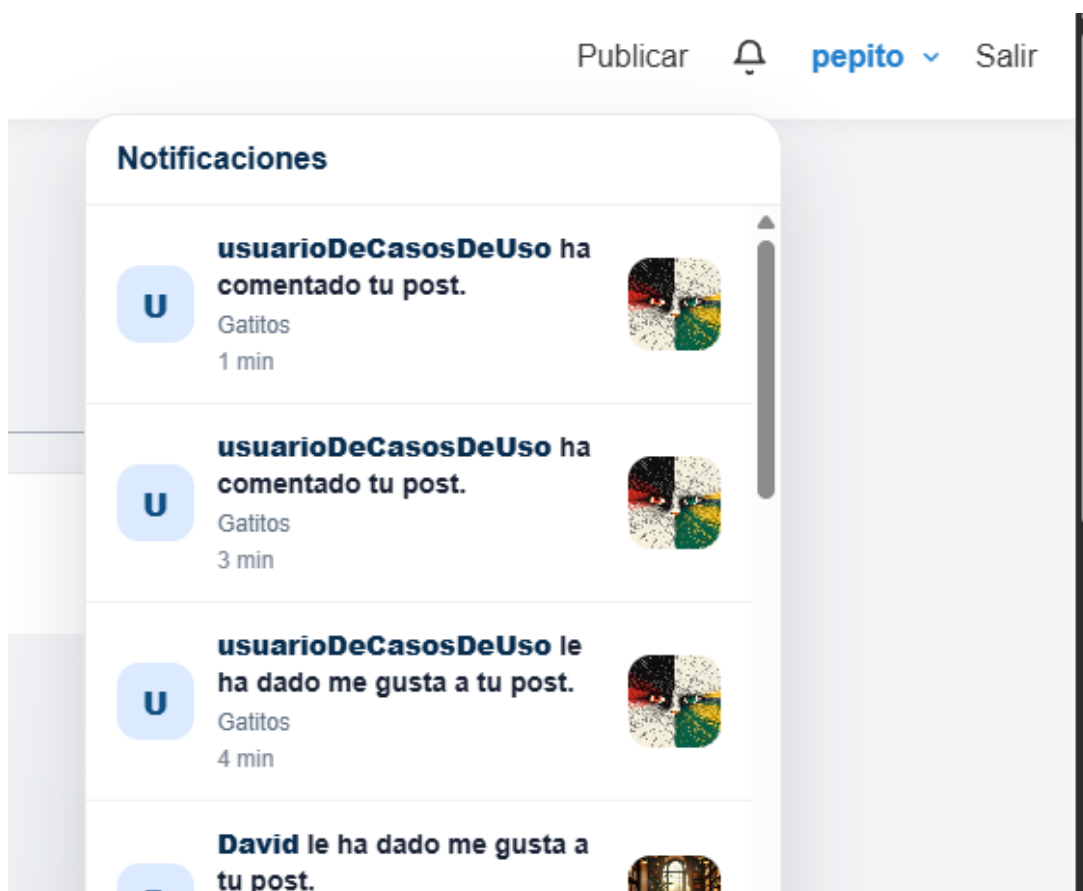
Este es un comentario respondiendo a otro comentario para los casos de uso

Responder

5. El usuario puede borrar un comentario creado previamente



- Postcondiciones: La interacción queda registrada y se genera una notificación para el autor de la publicación



Caso de uso 6: Seguir a un usuario (web)

- Actores: Usuario autenticado
- Precondiciones: El usuario ha iniciado sesión
- Flujo principal:
 1. El usuario accede al perfil de otro usuario y el sistema muestra la información del perfil y el botón de seguir

IA-Lovers

Inicio Explorar Siguiendo

Publicar 🔔 usuarioDeCasosDeUso ▾ Salir



superman120
Seguidores: 4 | Siguiendo: 1 | Posts: 2

Seguir



superman120

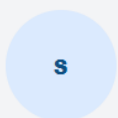
08:59 · 07/04/2026



La luna...
#Arte #Luna #Midjourney #Nasa

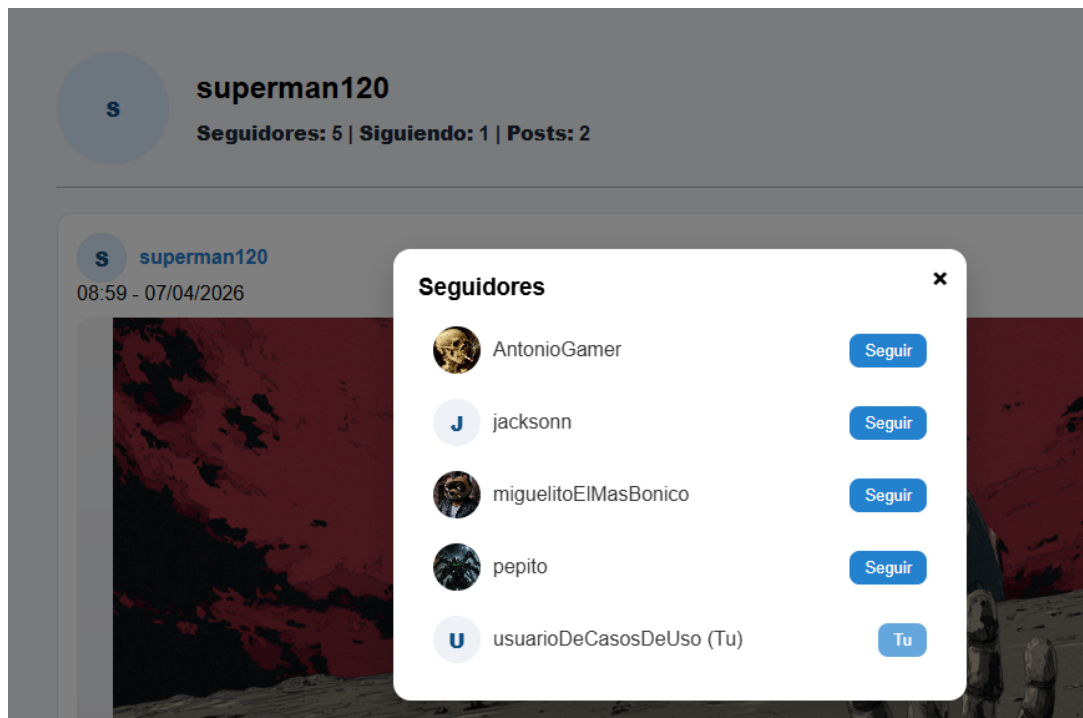
Activar Windows
Ve a Configuración para activar Windows.

2. El usuario pulsa el botón de seguir y el sistema registra el seguimiento y actualiza la interfaz

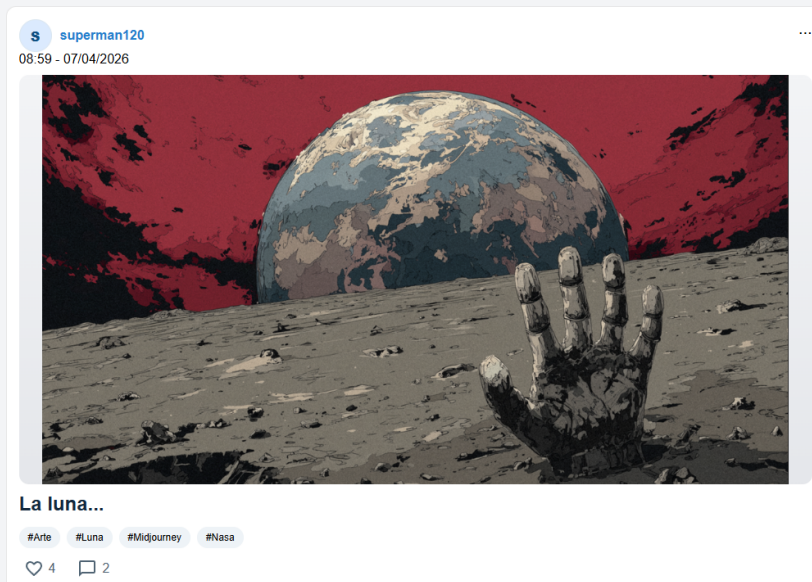


superman120
Seguidores: 5 | Siguiendo: 1 | Posts: 2

Dejar de seguir

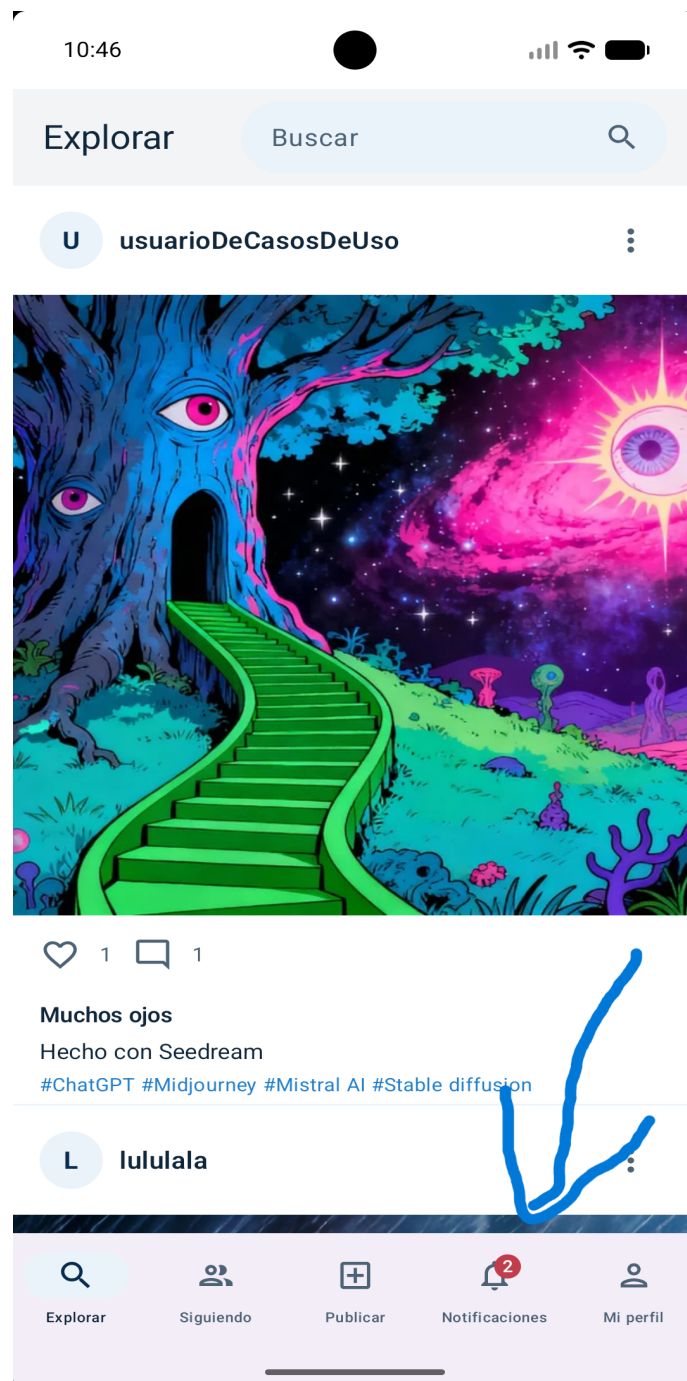


- Postcondiciones: El usuario sigue al otro usuario, y las publicaciones de este aparecen en su feed de "Siguiendo"



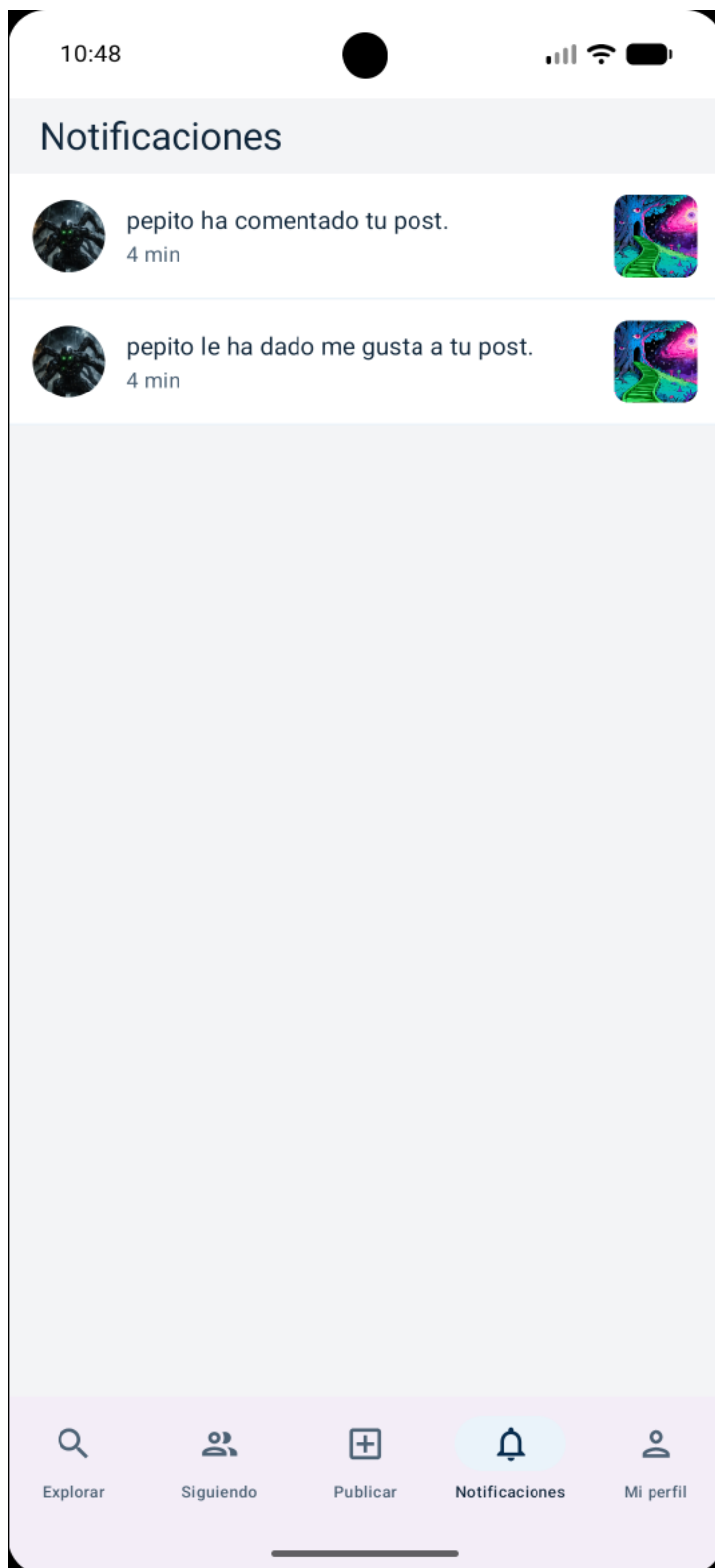
Caso de uso 7: Gestionar notificaciones (móvil)

- Actores: Usuario autenticado
- Precondiciones: El usuario ha iniciado sesión
- Flujo principal:
 1. El sistema muestra un contador de notificaciones no leídas en la barra de navegación

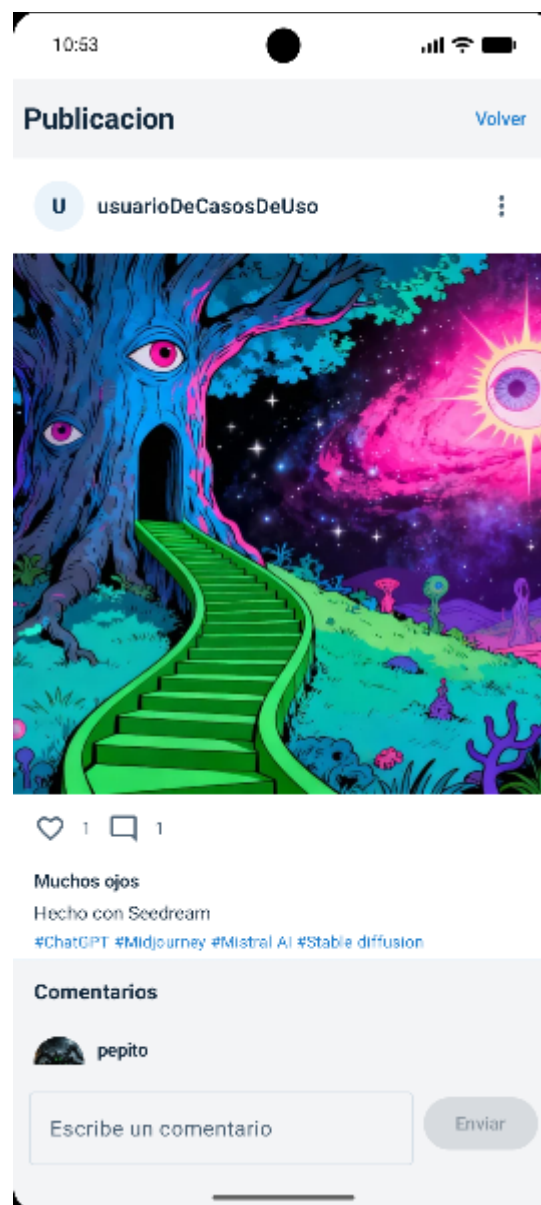
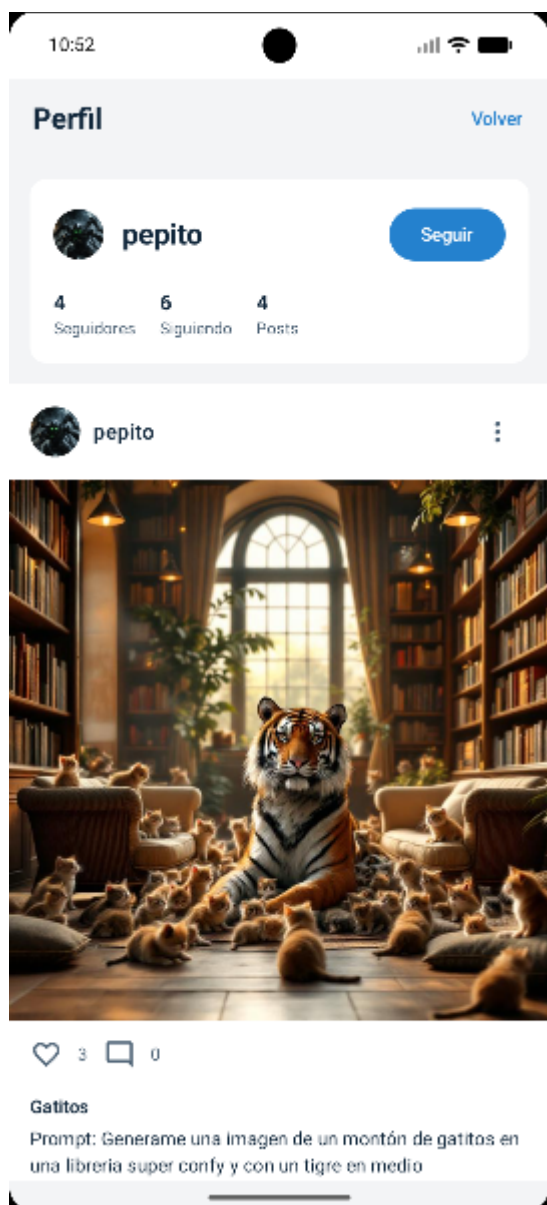




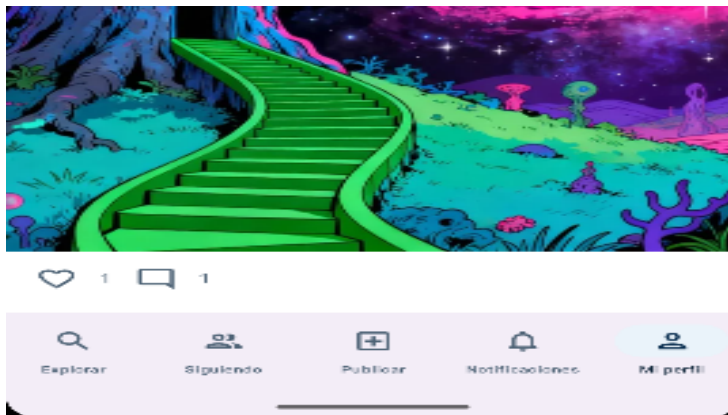
2. El usuario pulsa el icono de notificaciones y el sistema muestra el panel con la lista de notificaciones



- El usuario visualiza las notificaciones, que se marcan automáticamente como leídas, desde ahí puede ir directamente al usuario o post relacionado con la notificación

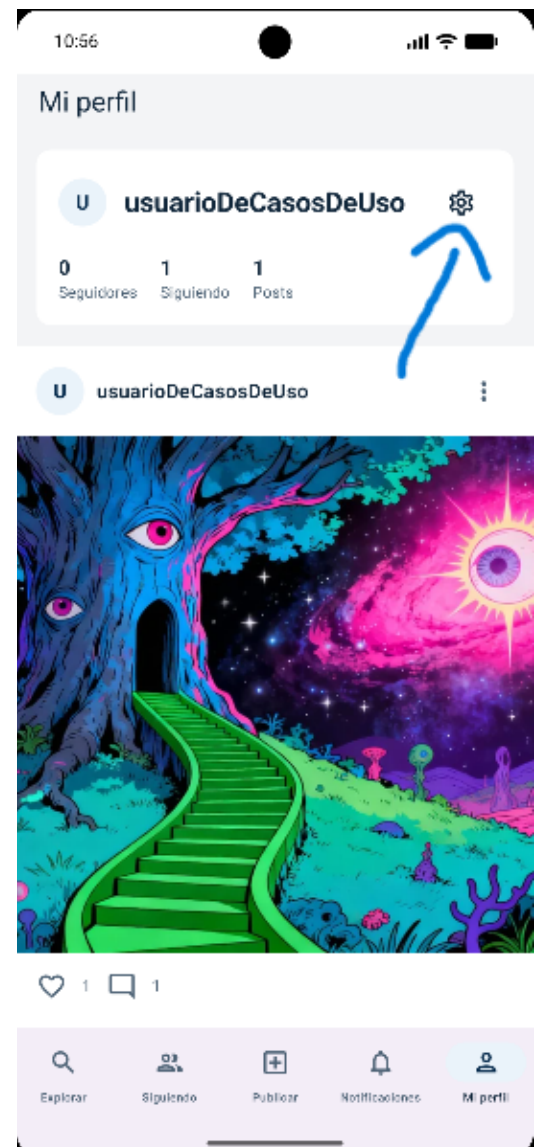


Postcondiciones: El contador de notificaciones se actualiza

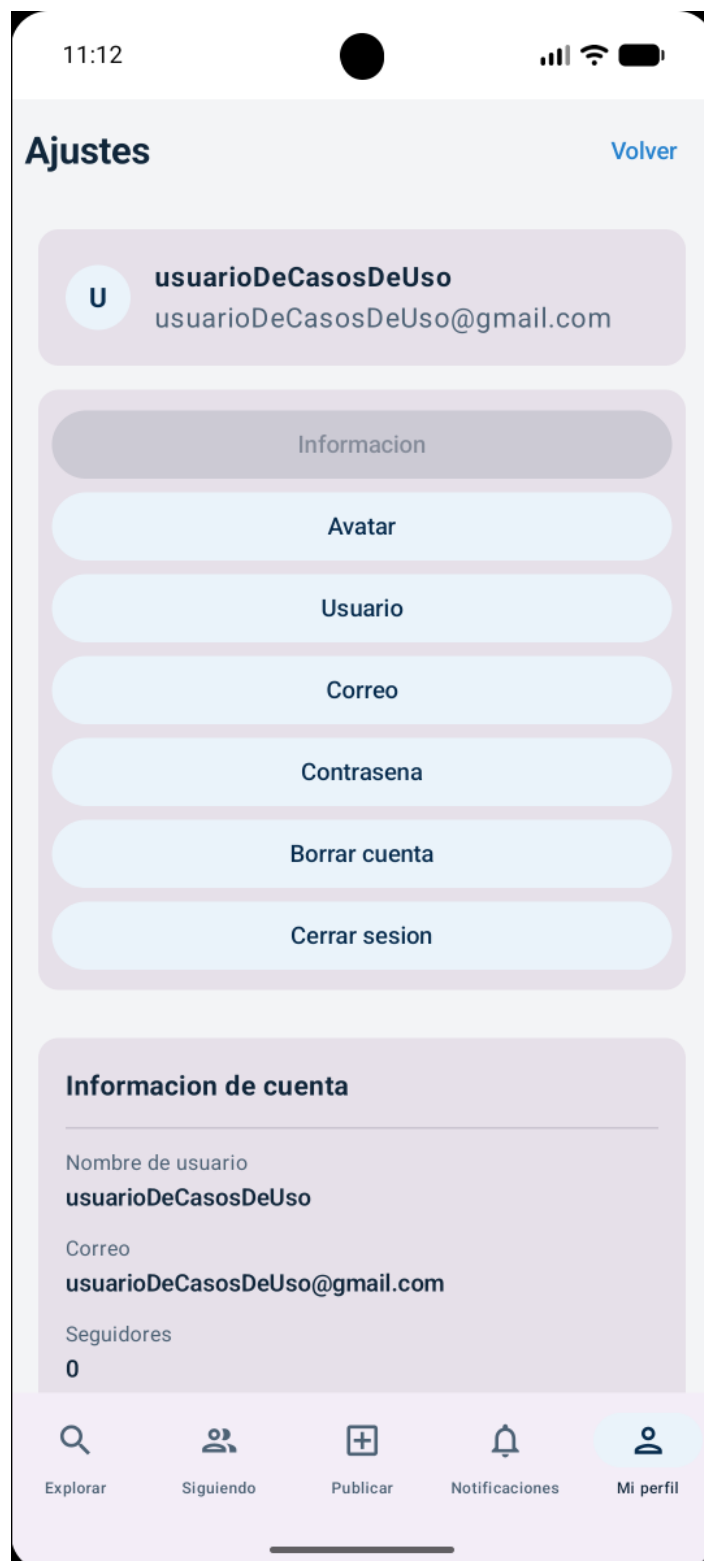


Caso de uso 8: Configurar perfil (móvil)

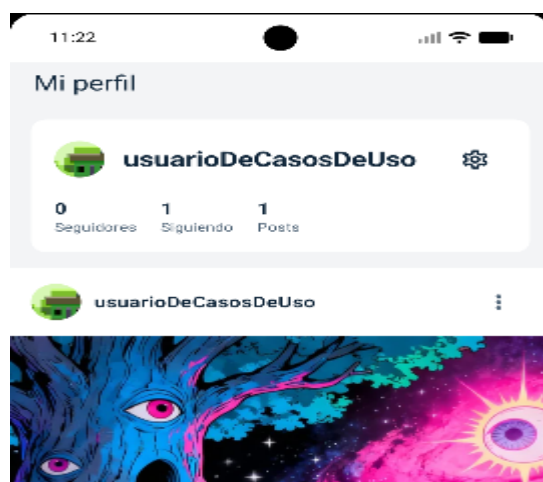
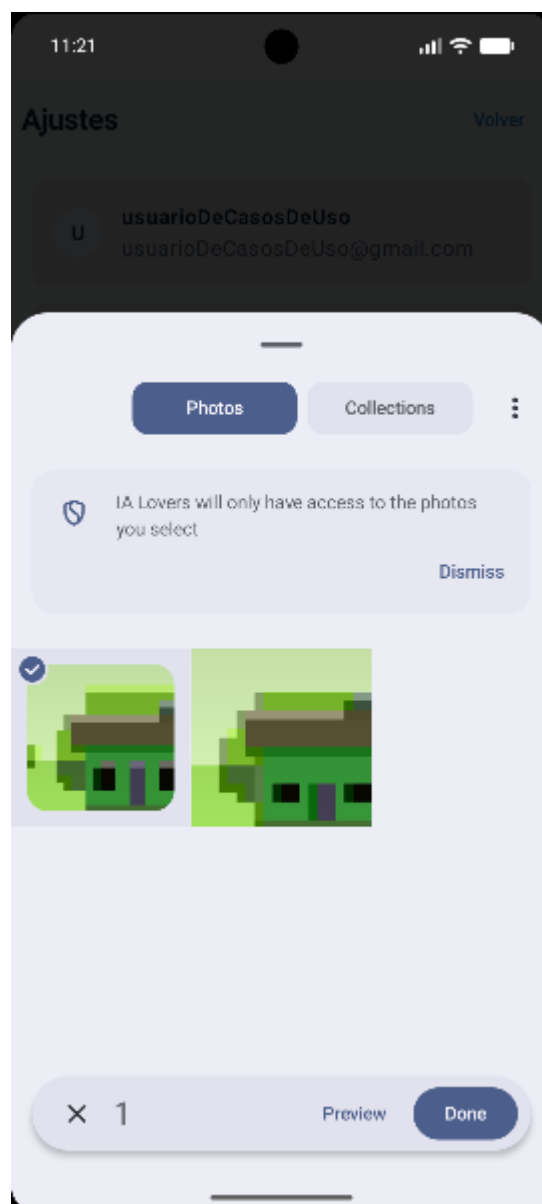
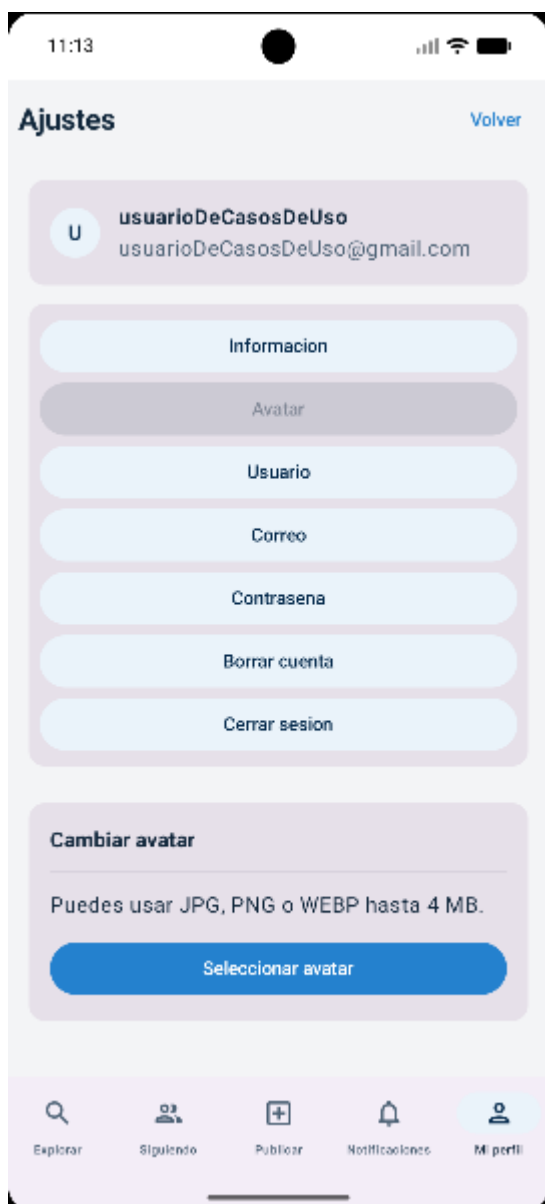
- Actores: Usuario autenticado
- Precondiciones: El usuario ha iniciado sesión
- Flujo principal:
 1. El usuario accede a la pantalla de ajustes ->

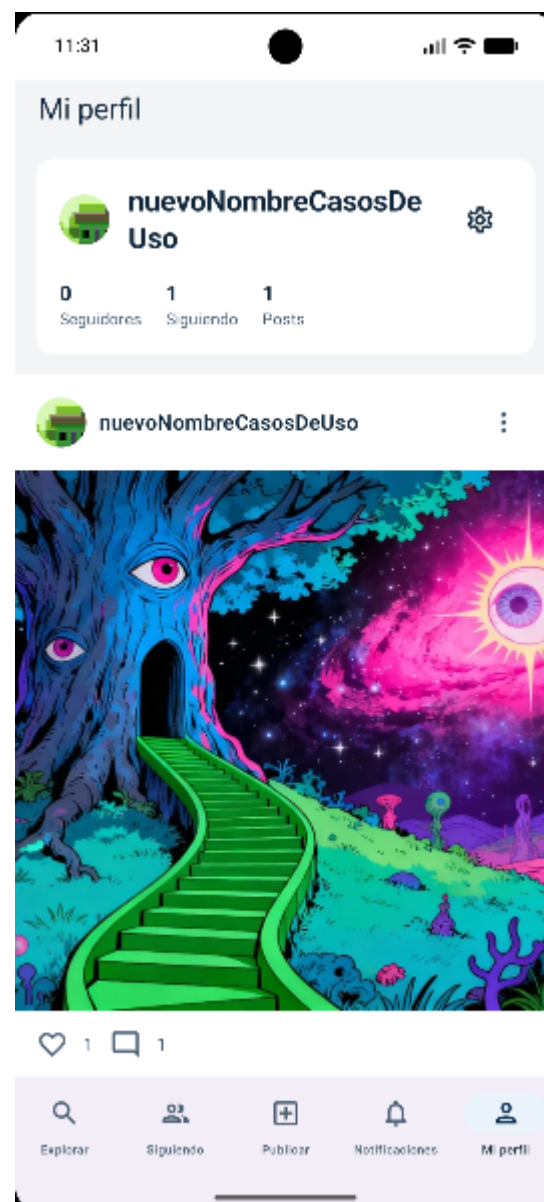
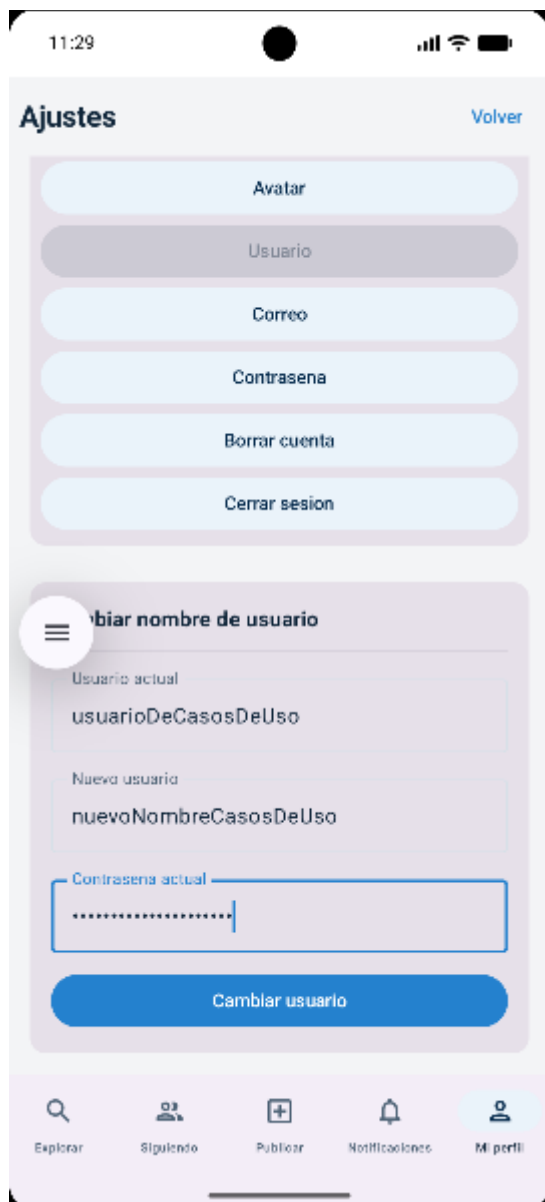


2. El sistema muestra las diferentes secciones configurables: información de cuenta, cambio de avatar, nombre de usuario, correo electrónico, contraseña, eliminación de cuenta y cerrar sesión.



3. El usuario selecciona una sección y realiza los cambios deseados







11:32

Ajustes

Volver

- Avatar
- Usuario
- Correo
- Contraseña
- Borrar cuenta
- Cerrar sesión

Cambiar correo

Correo actual
usuarioDeCasosDeUso@gmail.com

Nuevo correo

Contraseña actual

Enviar código

Explorar Siguiendo Publicar Notificaciones Mi perfil

11:32

Ajustes

Volver

- Avatar
- Usuario
- Correo
- Contraseña
- Borrar cuenta
- Cerrar sesión

Cambiar contraseña

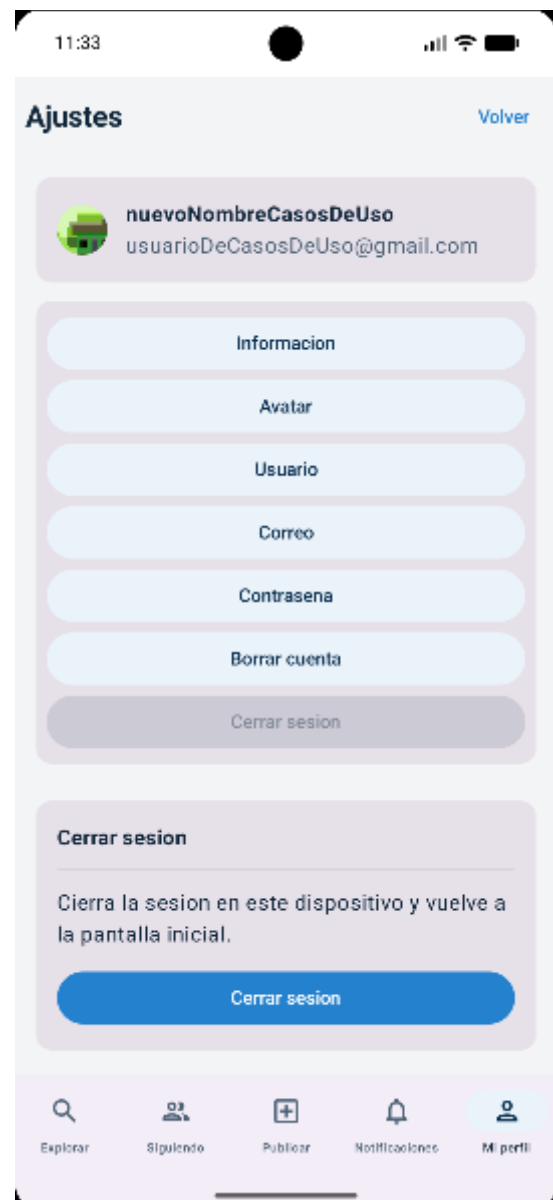
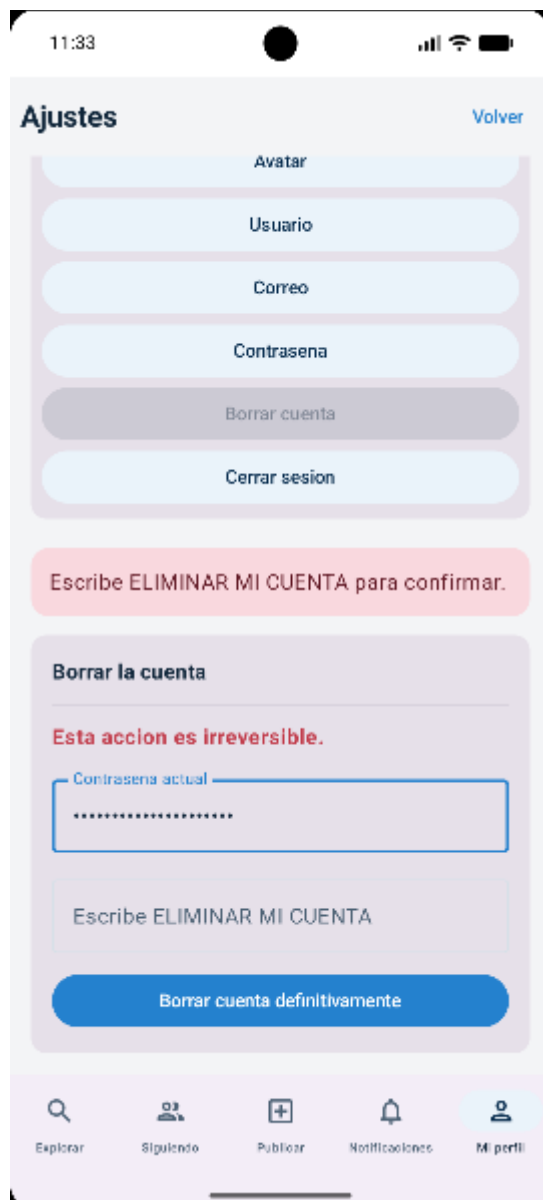
Contraseña actual

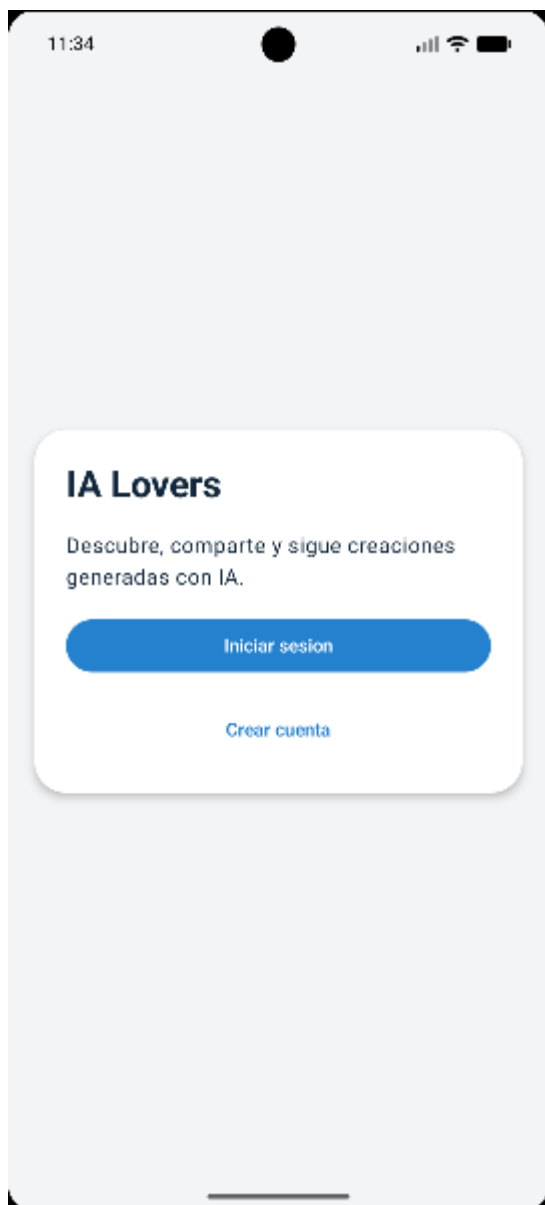
Nueva contraseña

Confirmar contraseña

Cambiar contraseña

Explorar Siguiendo Publicar Notificaciones Mi perfil





4. Si la operación requiere verificación, el sistema envía un código al correo
 5. El usuario confirma los cambios
- Postcondiciones: Los datos del perfil se actualizan en el sistema

4.3 Matriz de trazabilidad de requisitos

Tabla 9: Matriz de trazabilidad de requisitos funcionales

ID Requisito	Backend (ruta + archivo)	Implementación (Web)	Implementación (Android)
RF-01	<i>POST</i> /register/start (AuthController.php, PendingRegistration.php)	register.html + auth.js	AuthScreens.kt (RegisterScreen)
RF-02	<i>POST</i> /login (AuthController.php, User.php)	login.html + auth.js	AuthScreens.kt (LoginScreen)
RF-03	<i>POST</i> /password-reset/start (AuthController.php, PendingPasswordReset.php)	forgot_password.h tml	AuthScreens.kt
RF-04	<i>POST</i> /logout (AuthController.php, Auth.php)	header.js (performLogout)	MainScreen.kt
RF-05	<i>POST</i> /user/update (UserController.php, User.php)	profile_settings.ht ml + profile- settings.js	SettingsScreen.kt
RF-06	<i>POST</i> /user/avatar (UserController.php, Storage.php)	profile_settings.ht ml	SettingsScreen.kt



RF-07	POST /user/delete (UserController.php, Storage.php)	profile_settings.ht ml	SettingsScreen.kt
RF-08	POST /posts/create (PostController.php, Database.php, Storage.php)	create.html + posts.js kt	CreatePostScreen.
RF-09	POST /posts/delete (PostController.php, Database.php, Storage.php)	posts.js	PostCard.kt
RF-10	GET /posts (PostController.php, Database.php, Storage.php)	index.html, explore.html, posts.js	FeedScreen.kt
RF-11	GET /posts/show (PostController.php, Post.php)	post.html + posts.js kt	PostDetailScreen.
RF-12	POST /posts/toggle-like (PostController.php, Database.php, Storage.php)	posts.js	PostCard.kt
RF-13	GET /posts?q=&order= (PostController.php, Database.php, Storage.php)	explore.html	MainScreen.kt
RF-14	GET /posts?type=following (PostController.php, Database.php, Storage.php)	following.html	FeedScreen.kt



RF-15	POST /comments/create (CommentController.php, Comment.php)	post.html + posts.js kt	PostDetailScreen.
RF-16	POST /comments/delete (CommentController.php, Comment.php)	post.html + posts.js kt	PostDetailScreen.
RF-17	comments.parent_id (Comment.php)	posts.js (render anidado)	PostDetailScreen. kt
RF-18	POST /follow (FollowController.php)	user.html + posts.js	ProfileScreen.kt
RF-19	GET /users/username (UserController.php, User.php)	user.html	ProfileScreen.kt
RF-20	GET /users/followers, GET /users/following (FollowController.php)	profile.html (modal)	ProfileScreen.kt (dialog)
RF-21	Disparado desde PostController, CommentController, FollowController + Notification.php	Generación automática en backend	Generación automática en backend
RF-22	GET /notifications (NotificationController.php, Notification.php)	header.js (panel)	NotificationsScreen.kt



RF-23	POST /notifications/read (NotificationController.php, Notification.php)	header.js	NotificationsScreen.kt
RF-24	GET /notifications/unread-count (NotificationController.php, Notification.php)	header.js (badge)	MainScreen.kt (badge)
RF-25	GET /tags/search (TagController.php, Database.php)	create.html	CreatePostScreen.kt
RF-26	POST /tags/create (TagController.php)	create.html	CreatePostScreen.kt
RF-27	backend/core/*, backend/controllers/*, backend/models/*	header.js (apiUrl)	ApiService.kt + ApiFactory.kt
RF-28	—	CSS responsive + viewport meta	App nativa Jetpack Compose

Tabla 10: Matriz de trazabilidad de requisitos no funcionales

ID	Backend (ruta+archivo)	Web	Android
RNF-01	User.php, AuthController.php- password_hash(cost=12)	—	—
RNF-02	Auth.php random_bytes(32), TTL 90d	header.js (localStorage)	SessionStorage.kt
RNF-03	Todos los modelos — PDO prepared statements	—	—
RNF-04	UserController.php, PostController.php — finfo + 4MB	create.html (accept)	CreatePostScreen. kt
RNF-05	Altcha.php — HMAC- SHA256 challenge	Altcha-widget en formularios	—
RNF-06	RateLimiter.php — archivo + flock	—	—
RNF-07	api/index.php — cabeceras HTTP	—	—
RNF-08	—	CSS media queries	—
RNF-09	—	Tema azul #2481ce	Theme.kt (mismos colores)
RNF-10	Storage.php, Supabase Storage	—	ApiFactory.kt (Coil)



RNF-11	vercel.json — vercel-php@0.7.4	—	—
RNF-12	—	—	ApiFactory.kt (Retrofit + OkHttp)
RNF-13	Post.php — cursor-based paginación SQL	posts.js (initInfiniteScroll)	FeedScreen.kt (LazyColumn)



5. DISEÑO

5.1 Arquitectura del sistema

IA-Lovers se ha diseñado siguiendo una arquitectura cliente-servidor con API REST centralizada. Esta arquitectura separa la aplicación en varias capas independientes: clientes, backend, base de datos y almacenamiento de archivos. La decisión de utilizar una arquitectura de este tipo responde a la necesidad de ofrecer una experiencia multiplataforma, ya que tanto la aplicación web como la aplicación Android deben acceder a los mismos datos y ejecutar las mismas operaciones principales.

En lugar de que cada cliente implemente su propia lógica de negocio o acceda directamente a la base de datos, ambos se comunican con un backend común. Este backend actúa como capa intermedia entre las interfaces de usuario y los datos persistentes. De esta forma, operaciones como crear una publicación, iniciar sesión, comentar, seguir a un usuario o consultar notificaciones se realizan siempre mediante la API REST.

La estructura general del sistema se divide en los siguientes componentes:

- Aplicación web: desarrollada con HTML, CSS y JavaScript. Representa la versión accesible desde navegador. Se encarga de renderizar las páginas, capturar las acciones del usuario y enviar peticiones HTTP al backend.
- Aplicación Android: desarrollada con Kotlin y Jetpack Compose. Representa la versión móvil nativa. Utiliza Retrofit para comunicarse con la API REST y mantiene el estado de la interfaz mediante un ViewModel.
- Backend/API REST: desarrollado en PHP. Centraliza la lógica de negocio, valida las entradas, gestiona la autenticación, procesa las solicitudes y accede a la base de datos.
- Base de datos relacional: almacena la información estructurada del sistema: usuarios, publicaciones, comentarios, relaciones de seguimiento, “me gusta”, etiquetas, notificaciones y tokens de sesión.

- Supabase Storage: almacena los archivos subidos por los usuarios, como imágenes de publicaciones y avatares. La base de datos no guarda el archivo binario, sino la ruta necesaria para recuperarlo.

La comunicación entre los clientes y el backend se realiza mediante HTTP, utilizando principalmente respuestas en formato JSON. Este formato es adecuado porque es ligero, fácil de interpretar tanto desde JavaScript como desde Kotlin y ampliamente utilizado en APIs REST.

La arquitectura puede representarse de la siguiente manera:

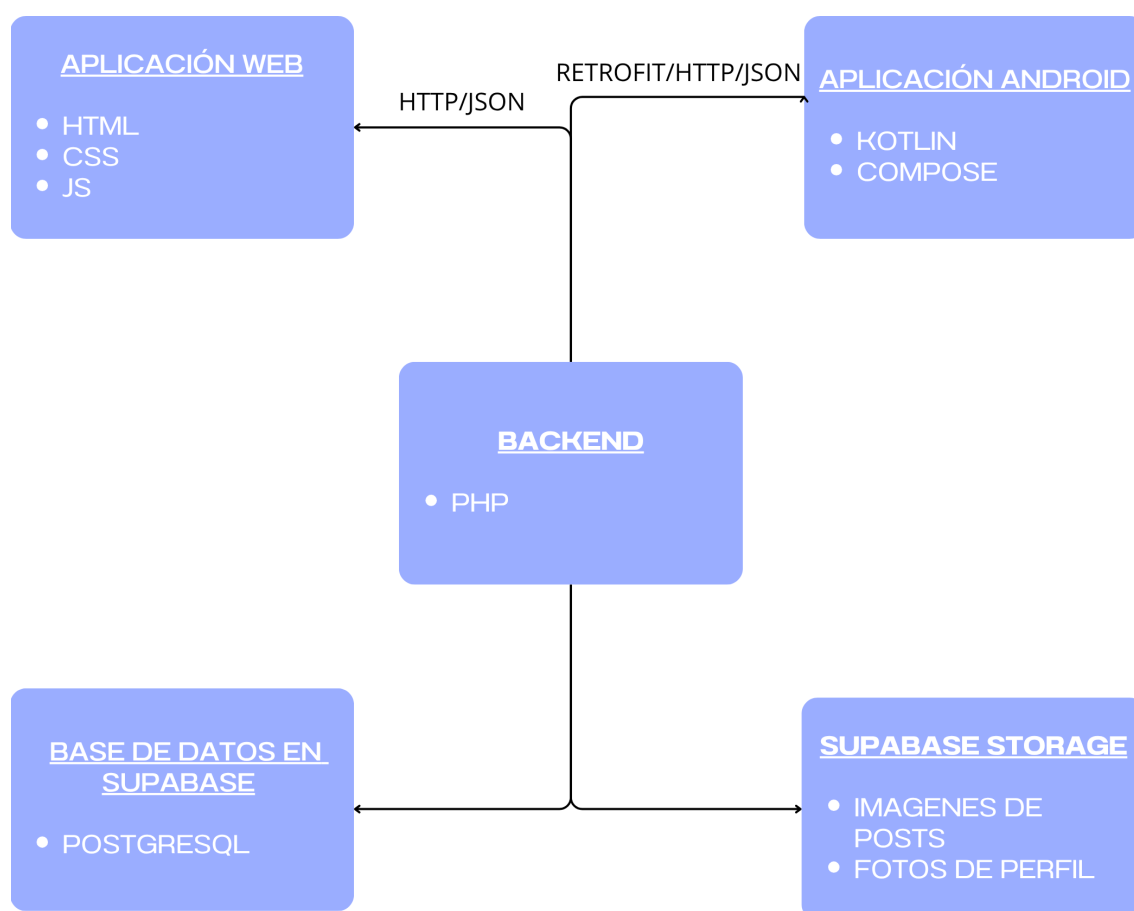


Figura 1: Arquitectura de los componentes



5.1.1 Diseño de la aplicación web

La aplicación web se ha desarrollado con HTML, CSS y JavaScript. HTML define la estructura de las páginas, CSS controla la presentación visual y JavaScript aporta interactividad, por ejemplo se utiliza para enviar peticiones a la API, renderizar publicaciones dinámicamente, gestionar el scroll infinito, actualizar botones de “me gusta”, mostrar notificaciones y controlar formularios de autenticación o ajustes.

Las páginas principales son:

- index.html: pantalla principal.
- explore.html: exploración de publicaciones con búsqueda y ordenación.
- following.html: feed de usuarios seguidos.
- create.html: creación de publicaciones.
- post.html: detalle de una publicación.
- profile.html: perfil del usuario autenticado.
- user.html: perfil público de otro usuario.
- profile_settings.html: ajustes de cuenta.
- login.html, register.html y forgot_password.html: flujos de autenticación.



La lógica de publicaciones se concentra en posts.js. Este archivo renderiza tarjetas de publicaciones, gestiona el scroll infinito, controla los “me gusta”, redirige al detalle de publicación y permite eliminar publicaciones propias. El uso de JavaScript permite que el feed se cargue dinámicamente sin tener que generar todo el contenido desde el servidor.

El archivo header.js se encarga de la navegación común, el estado de sesión y el panel de notificaciones. Esto evita repetir la lógica de cabecera en cada página y permite que el contador de notificaciones se comporte de forma coherente.

La web utiliza localStorage para guardar el token y la información básica del usuario. Esta decisión permite mantener la sesión entre recargas del navegador. Cuando se realiza una petición protegida, JavaScript añade el token en la cabecera Authorization.

El diseño responsive se gestiona mediante CSS y media queries. Esto permite adaptar la interfaz a distintos tamaños de pantalla, aunque la versión Android ofrece una experiencia móvil más nativa.



5.1.2 Diseño de la aplicación Android

En la aplicación Android se ha utilizado Kotlin como lenguaje principal. Kotlin es el lenguaje recomendado actualmente para el desarrollo Android moderno, ya que ofrece una sintaxis más segura y expresiva que Java, mejor integración con las APIs actuales de Android y soporte nativo para corrutinas. Las corrutinas permiten ejecutar operaciones asíncronas, como peticiones de red, sin bloquear la interfaz de usuario.

Jetpack Compose se ha utilizado para construir la interfaz gráfica de Android. Compose es el sistema moderno de interfaces declarativas de Android. A diferencia del enfoque clásico basado en archivos XML, Compose permite definir la interfaz directamente mediante funciones Kotlin. Esto facilita que la pantalla se actualice automáticamente cuando cambia el estado de la aplicación. Por ejemplo, si el contador de notificaciones cambia o si se actualiza el estado de un “me gusta”, Compose puede recomponer la parte afectada de la interfaz sin reconstruir manualmente toda la pantalla.

Retrofit se utiliza como cliente HTTP en Android. Su función es transformar métodos Kotlin en peticiones HTTP a la API REST. Por ejemplo, una función como toggleLike(), que sirve para dar un “me gusta”, definida en ApiService se convierte internamente en una petición POST al endpoint correspondiente. Retrofit simplifica la comunicación con APIs porque permite declarar rutas, parámetros, cuerpos JSON y peticiones multipart de forma estructurada. Además, se integra con OkHttp, que se encarga de la ejecución real de las conexiones HTTP.

OkHttp actúa como cliente de red de bajo nivel. Gestiona conexiones, cabeceras, interceptores y envío de peticiones. En este proyecto se utiliza junto con Retrofit para incluir tokens de autenticación y configurar la comunicación con el backend.

La aplicación Android sigue una arquitectura basada en estado. El componente central es AppViewModel, que actúa como intermediario entre la interfaz y la API. El



ViewModel mantiene el estado de la aplicación, ejecuta llamadas de red y expone los datos que las pantallas deben mostrar.

Este diseño es adecuado porque en Android los cambios de configuración, como rotación de pantalla o recreación de actividades, pueden destruir y reconstruir partes de la interfaz. Utilizar un ViewModel permite conservar el estado de forma más estable.

La navegación principal se basa en estados definidos en la aplicación:

- Splash: pantalla inicial mientras se comprueba si hay sesión.
- AuthChoice: pantalla de acceso a login o registro.
- Login: inicio de sesión.
- Register: registro y verificación.
- Main: zona principal autenticada.

Dentro de Main, la aplicación organiza las funcionalidades en pestañas:

- Explore: feed general.
- Following: feed de usuarios seguidos.
- Create: creación de publicación.
- Notifications: notificaciones.
- Profile: perfil propio.

El archivo ApiService.kt define la interfaz de comunicación con el backend mediante Retrofit. Cada método representa un endpoint de la API. Por ejemplo, posts() consulta publicaciones, createPost() envía una imagen mediante multipart, toggleLike() cambia el



estado de un “me gusta” y `notificationsUnreadCount()` recupera el número de notificaciones no leídas.

La aplicación utiliza modelos de datos definidos en Kotlin para representar las respuestas del backend. Esto mejora la seguridad del código, ya que cada respuesta esperada tiene una estructura concreta. Si una publicación contiene `id`, `title`, `filePath`, `likesCount` y `likedByUser`, esos datos se representan como propiedades tipadas.

Para la carga de imágenes se utiliza Coil. Coil es una biblioteca de carga de imágenes para Android basada en Kotlin. Permite descargar, cachear y mostrar imágenes desde URLs de forma eficiente dentro de componentes Compose. Esto es especialmente importante en una red social visual, donde el feed puede contener muchas imágenes.

5.1.3 Diseño del backend

El backend se ha implementado en PHP. PHP es una tecnología ampliamente utilizada para aplicaciones web y APIs, especialmente en entornos con servidores Apache o despliegues compatibles. En este proyecto, PHP se organiza en controladores, modelos y clases auxiliares, siguiendo una separación de responsabilidades similar al patrón MVC, aunque adaptada a una API REST.

El backend se estructura alrededor de un punto de entrada principal, `backend/index.php`. Este archivo configura las cabeceras HTTP de seguridad, define el tipo de respuesta JSON, habilita los métodos permitidos y registra las rutas de la API.

El sistema utiliza un router propio. La clase Router recibe el método HTTP y la URI solicitada, normaliza la ruta y ejecuta la función asociada. Esto permite mantener las rutas centralizadas y evitar lógica dispersa. Por ejemplo:

```
GET /posts          -> PostController::feed()

GET /posts/show     -> PostController::show()

POST /posts/create  -> PostController::create()

POST /posts/toggle-like -> PostController::toggleLike()

POST /comments/create -> CommentController::create()

POST /follow        -> FollowController::follow()
```

Los controladores representan la capa de lógica de aplicación. Cada controlador se encarga de un área funcional concreta:

`AuthController` gestiona registro, login, recuperación de contraseña, verificación por correo y cierre de sesión.

`PostController` gestiona publicaciones, feed, detalle, eliminación, subida de imágenes, etiquetas y “me gusta”.

`CommentController` gestiona comentarios y respuestas anidadas.



UserController gestiona perfiles, edición de cuenta, avatar, correo, contraseña y eliminación de cuenta.

FollowController gestiona seguidores y seguidos.

NotificationController gestiona la recuperación y lectura de notificaciones.

TagController gestiona búsqueda y creación de etiquetas.

Esta separación facilita el mantenimiento, ya que cada archivo tiene una responsabilidad clara. Si se necesita modificar el sistema de comentarios, se puede trabajar principalmente sobre CommentController y el modelo relacionado sin afectar directamente a la autenticación o las publicaciones.

Además de los controladores, existen clases auxiliares importantes:

Auth se encarga de emitir, validar, renovar y revocar tokens de sesión.

Middleware protege rutas que requieren usuario autenticado.

Storage gestiona la subida, eliminación y transformación de rutas de imágenes en URLs públicas.

Response centraliza las respuestas JSON.

RateLimiter limita intentos repetidos en endpoints sensibles.

Alcha genera y valida desafíos anti-bots.

GmailMailer permite enviar códigos de verificación mediante correo electrónico.

La autenticación se basa en tokens opacos. Cuando un usuario inicia sesión correctamente, el backend genera un token aleatorio mediante `random_bytes(32)` y lo almacena en la tabla `user_tokens`. Este token se envía al cliente y posteriormente se incluye en las peticiones protegidas mediante la cabecera `Authorization: Bearer`. Esta solución evita enviar usuario y contraseña en cada petición y permite revocar sesiones de forma individual.

5.1.4 Diseño de la base de datos

La base de datos se ha diseñado siguiendo un modelo relacional normalizado. La normalización permite reducir duplicidades, representar relaciones de forma clara y mantener la integridad de los datos mediante claves primarias y foráneas.

Se puede ver aquí (hacer zoom para ver bien):

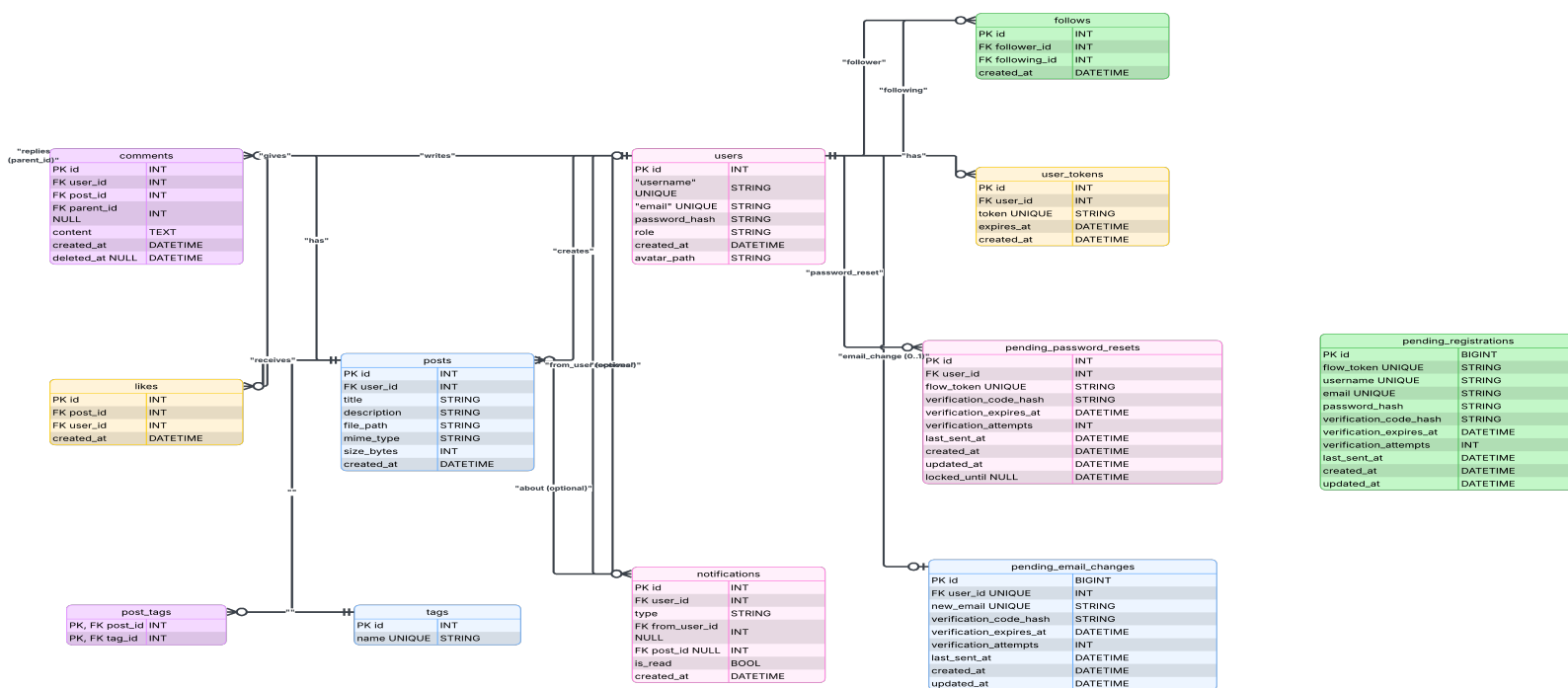


Figura 2: Diagrama de la base de datos



La tabla clave del sistema es users, ya que la mayoría de entidades dependen de un usuario. Esta tabla almacena los datos principales de la cuenta: identificador, nombre de usuario, correo electrónico, hash de contraseña, rol, fecha de creación y ruta del avatar. El nombre de usuario y el correo son únicos para evitar cuentas duplicadas y permitir una identificación clara dentro de la plataforma. La contraseña no se almacena en texto plano, sino como password_hash, calculado con bcrypt.

La tabla posts representa las publicaciones. Cada publicación pertenece a un usuario mediante user_id, que referencia a users(id). Se almacenan el título, descripción, ruta del archivo, tipo MIME, tamaño del archivo y fecha de creación. La imagen no se guarda directamente en la tabla, sino en Supabase Storage. La columna file_path contiene la referencia necesaria para construir la URL pública. Esta separación evita sobrecargar la base de datos con archivos binarios y facilita servir imágenes desde un sistema especializado.

La tabla comments almacena los comentarios de las publicaciones. Cada comentario tiene un user_id, un post_id y un contenido. Además, incluye parent_id, que referencia a la propia tabla comments. Esta relación autorreferenciada permite implementar comentarios anidados, es decir, respuestas a otros comentarios. También se incluye deleted_at, lo que permite aplicar borrado lógico en ciertos casos. Esto es útil cuando un comentario tiene respuestas: en lugar de eliminarlo físicamente y romper la estructura del hilo, se puede marcar como eliminado y conservar sus respuestas.

La tabla likes representa la relación entre usuarios y publicaciones que les han gustado. Tiene user_id, post_id y fecha de creación. Se ha diseñado como tabla independiente porque la relación entre usuarios y publicaciones es de muchos a muchos: un usuario puede dar “me gusta” a muchas publicaciones, y una publicación puede recibir “me gusta” de muchos usuarios. Separar esta información permite consultar recuentos, comprobar si un usuario ya ha dado like y eliminar la interacción si vuelve a pulsar el botón.



La tabla `follows` representa el seguimiento entre usuarios. Incluye `follower_id`, que corresponde al usuario que sigue, y `following_id`, que corresponde al usuario seguido. Esta relación también es de muchos a muchos dentro de la misma entidad `users`. Se utiliza una tabla separada porque un usuario puede seguir a muchos usuarios y, al mismo tiempo, ser seguido por muchos otros. Esta estructura permite construir feeds personalizados, como el feed “Siguiendo”.

La tabla `notifications` almacena eventos relevantes para el usuario. Incluye `user_id`, que indica quién recibe la notificación, `type`, que indica el tipo de evento, `from_user_id`, que indica quién originó la acción, `post_id`, si la notificación está asociada a una publicación, `is_read` y `created_at`. Esta estructura flexible permite representar notificaciones de distintos tipos, como nuevos seguidores, “me gusta”, comentarios o respuestas.

La tabla `tags` contiene las etiquetas disponibles. Cada etiqueta tiene un nombre único. Se ha separado en una tabla propia para evitar duplicación de texto y permitir búsquedas más eficientes. Por ejemplo, si muchas publicaciones usan la etiqueta “Anime”, no se repite el texto completo como dato independiente en cada publicación.

La tabla `post_tags` conecta publicaciones y etiquetas. Su clave primaria compuesta está formada por `post_id` y `tag_id`. Esta decisión impide que una misma etiqueta se asocie dos veces a la misma publicación. La relación entre publicaciones y etiquetas es de muchos a muchos: una publicación puede tener varias etiquetas y una etiqueta puede aparecer en muchas publicaciones. Por eso se utiliza una tabla intermedia.



La tabla `user_tokens` almacena las sesiones activas. Cada token pertenece a un usuario y tiene una fecha de expiración. Esta tabla permite mantener varias sesiones por usuario, por ejemplo una en web y otra en Android. También permite cerrar sesión eliminando el token correspondiente o invalidar sesiones antiguas cuando sea necesario.

Las tablas `pending_registrations`, `pending_password_resets` y `pending_email_changes` representan procesos temporales de verificación. Se han separado de la tabla `users` porque todavía no son datos definitivos o porque pertenecen a flujos sensibles que deben caducar. Por ejemplo, durante el registro, el usuario todavía no debe existir como cuenta activa hasta que introduzca correctamente el código enviado por correo. Por eso se almacena temporalmente en `pending_registrations`, junto con un `flow_token`, el hash del código de verificación, la fecha de expiración y el número de intentos.

En el caso de recuperación de contraseña, `pending_password_resets` permite controlar el proceso sin modificar la contraseña directamente hasta que el usuario demuestre tener acceso al correo. La columna `locked_until` permite bloquear temporalmente el flujo si se superan los intentos permitidos.

En el cambio de correo, `pending_email_changes` almacena el nuevo correo pendiente de confirmar. Se utiliza una restricción `UNIQUE` sobre `user_id`, de modo que cada usuario solo pueda tener un cambio de correo pendiente al mismo tiempo. También se aplica unicidad sobre `new_email` para evitar que dos usuarios intenten confirmar el mismo correo simultáneamente.



5.1.5 Diseño del almacenamiento de imágenes en Supabase Storage

Aunque la base de datos almacena la información estructurada de la plataforma, las imágenes de publicaciones y los avatares de usuario se guardan en Supabase Storage. Esta decisión se ha tomado porque las imágenes son archivos binarios relativamente pesados, mientras que la base de datos relacional está mejor orientada al almacenamiento y consulta de datos estructurados como usuarios, publicaciones, comentarios o relaciones sociales.

La estructura del almacenamiento se organiza dentro de un bucket de Supabase mediante una carpeta principal llamada uploads. Dentro de esta carpeta se crea una subcarpeta por cada usuario, utilizando su identificador interno como nombre de carpeta. De esta forma, los archivos quedan organizados siguiendo el patrón:

uploads/

1/

imagen_publicacion_1.webp

avatar_usuario.jpg

2/

imagen_publicacion_2.png

3/

avatar_usuario.webp

En términos generales, la ruta de un archivo sigue la forma:

uploads/{id_usuario}/{nombre_archivo}

Por ejemplo, si el usuario con identificador 5 sube una imagen, esta se almacena en:

uploads/5/a83f29c0.webp



La base de datos no guarda la imagen como tal, sino únicamente la referencia al archivo mediante campos como `file_path` en la tabla `posts` o `avatar_path` en la tabla `users`. Posteriormente, el backend utiliza la clase `Storage` para construir la URL pública correspondiente a partir del identificador del usuario y el nombre del archivo almacenado.

Esta organización por carpetas de usuario tiene varias ventajas. En primer lugar, facilita mantener los archivos ordenados y asociados a su propietario. Si un usuario elimina su cuenta, el sistema puede localizar con facilidad todos los archivos que le pertenecen dentro de su carpeta y eliminarlos del almacenamiento. En segundo lugar, evita que todos los archivos se acumulen en un único directorio, lo que mejora la claridad de la estructura y simplifica tareas de mantenimiento. En tercer lugar, permite generar nombres de archivo aleatorios sin depender del nombre original subido por el usuario, reduciendo conflictos entre archivos con el mismo nombre.

Guardar las imágenes en Supabase Storage en lugar de almacenarlas directamente dentro de la base de datos ofrece varias ventajas técnicas. La primera es el rendimiento: las bases de datos relacionales están optimizadas para consultar registros, aplicar filtros, ordenar datos y mantener relaciones, pero no son la opción más eficiente para servir archivos multimedia pesados. Si las imágenes se guardaran como datos binarios dentro de la base de datos, cada consulta podría volverse más pesada y el tamaño de la base crecería rápidamente.

La segunda ventaja es la escalabilidad. Un sistema de almacenamiento de objetos como Supabase Storage está diseñado específicamente para gestionar archivos. Esto permite almacenar muchas imágenes sin afectar directamente al rendimiento de las consultas SQL. La base de datos solo conserva metadatos ligeros, como la ruta, tipo MIME, tamaño y usuario propietario.

La tercera ventaja es la separación de responsabilidades. La base de datos se encarga de la información estructurada y relacional, mientras que Supabase Storage se encarga de los archivos. Esta separación hace que el sistema sea más mantenible y permite modificar la gestión de imágenes sin alterar el modelo principal de datos.



La cuarta ventaja está relacionada con la entrega de contenido. Al trabajar con URLs públicas o firmadas, las imágenes pueden ser servidas directamente desde el sistema de almacenamiento, sin que el backend tenga que leer el archivo y reenviarlo manualmente en cada petición. Esto reduce carga en el servidor PHP y mejora la experiencia de usuario al cargar feeds con muchas imágenes.

Por tanto, la tabla posts almacena datos como title, description, mime_type, size_bytes y file_path, pero el contenido físico de la imagen permanece en Supabase Storage. Del mismo modo, la tabla users guarda avatar_path, pero la imagen del avatar se almacena dentro de la carpeta del usuario en uploads/{id_usuario}/.



5.2 Interacción y Flujo de la Aplicación

El flujo general de IA-Lovers comienza con un usuario no autenticado. Este usuario puede explorar contenido público, visualizar publicaciones y consultar perfiles. Sin embargo, cuando intenta realizar una acción social, como dar “me gusta”, comentar, publicar o seguir a otro usuario, el sistema solicita autenticación.

Durante el registro, el usuario introduce nombre de usuario, correo y contraseña. El backend valida los datos, comprueba que el correo y el nombre de usuario no estén en uso y crea un registro temporal en `pending_registrations`. Después envía un código de verificación al correo. El usuario debe introducir ese código para completar el registro. Esta decisión evita crear cuentas no verificadas y reduce el riesgo de registros automatizados.

En el inicio de sesión, el backend compara la contraseña introducida con el hash almacenado. Si es correcta, genera un token de sesión y lo guarda en `user_tokens`. A partir de ese momento, el cliente utiliza ese token para identificarse en las operaciones protegidas.

El flujo de publicación consta de varias fases. Primero, el cliente selecciona una imagen y envía un formulario multipart al backend. El backend valida que el archivo exista, que no supere el tamaño máximo, que el tipo MIME real sea JPEG, PNG o WebP y que el título y descripción cumplan los límites establecidos. Después sube la imagen a Supabase Storage y guarda la publicación en la base de datos. Si se indican etiquetas, el sistema busca si ya existen; si no existen, las crea y las asocia mediante `post_tags`.



El flujo del feed utiliza paginación basada en cursor. En lugar de cargar todas las publicaciones, el backend devuelve un bloque limitado y un cursor para solicitar la siguiente página. Esta técnica mejora el rendimiento y evita consultas demasiado pesadas. En la web, el scroll infinito se implementa con IntersectionObserver, que detecta cuándo el usuario se aproxima al final del feed. En Android, el comportamiento equivalente se integra con listas de Compose.

El flujo de interacción social se basa en operaciones atómicas. Cuando un usuario pulsa “me gusta”, el backend comprueba si ya existe un registro en likes. Si existe, lo elimina; si no existe, lo crea. Además, si la acción afecta a una publicación de otro usuario, se genera o elimina la notificación correspondiente. Lo mismo ocurre con comentarios, respuestas y seguimientos.



5.3 Diseño de la interfaz gráfica

La interfaz de IA-Lovers se ha diseñado priorizando el contenido visual, ya que el objetivo principal de la plataforma es compartir imágenes generadas mediante inteligencia artificial. Por este motivo, las imágenes tienen un papel central en el feed y en el detalle de publicación.

La identidad visual utiliza tonos azules, asociados a tecnología, inteligencia artificial y confianza. En la web se utilizan colores como #2481ce, #154c79 y #072c4e, junto con fondos claros y superficies blancas. En Android, estos colores se trasladan al tema definido en Theme.kt, manteniendo coherencia entre plataformas.

La navegación web se basa en una barra superior fija. Esta barra permite acceder a las secciones principales sin perder orientación: explorar, siguiendo, crear publicación, notificaciones y perfil. La decisión de fijar la navegación mejora la accesibilidad en pantallas largas, especialmente cuando el usuario navega por feeds con scroll infinito.

En Android se utiliza una navegación adaptada al móvil, con pestañas principales y pantallas Compose. Esta decisión responde a las convenciones de diseño móvil, donde el usuario espera acceder rápidamente a las secciones principales mediante controles inferiores o navegación persistente.

La experiencia de usuario se apoya en validaciones tempranas, mensajes claros y actualización inmediata de la interfaz. Por ejemplo, al pulsar “me gusta”, la interfaz puede actualizar el contador de forma optimista y corregirlo si el backend devuelve un error. Esta técnica mejora la sensación de fluidez, aunque el dato definitivo siempre procede del servidor.



6. HERRAMIENTAS UTILIZADAS

6.1 Herramientas Hardware

Para el desarrollo principal del proyecto se ha utilizado un ordenador de sobremesa personal. En este equipo se ha programado la aplicación web, el backend y la aplicación Android, además de realizar pruebas locales mediante un servidor Apache y una base de datos en entorno de desarrollo.

También se ha utilizado un dispositivo móvil Xiaomi con sistema operativo Android para probar la aplicación móvil en un entorno real. Estas pruebas han permitido comprobar aspectos que no siempre se aprecian correctamente en un emulador, como el rendimiento general, la instalación de APKs, la navegación táctil, la carga de imágenes y el comportamiento de la interfaz en una pantalla física.

Además, durante el desarrollo de la aplicación Android se han utilizado emuladores incluidos en Android Studio. Estos emuladores han permitido probar la aplicación en distintos tamaños de pantalla y versiones del sistema Android sin depender exclusivamente del dispositivo físico.

6.2 Herramientas Software

Para el desarrollo local se ha utilizado XAMPP, que proporciona un entorno con Apache, PHP y herramientas asociadas. Al inicio del proyecto también se utilizó MySQL/phpMyAdmin para las primeras pruebas de base de datos y gestión visual de tablas. Posteriormente, el proyecto pasó a utilizar Supabase en producción.

El backend y la aplicación web se han desarrollado con VSCodium como editor de código, ya que no trae IA integrada y me interesaba aprender. En esta parte del proyecto se han utilizado PHP para la API REST, HTML para la estructura de las páginas web, CSS para el diseño visual y JavaScript para la lógica del cliente, peticiones a la API, renderizado dinámico del feed, gestión de sesión e interacciones.



La aplicación Android se ha desarrollado con Android Studio, utilizando Kotlin y Jetpack Compose. Kotlin se ha empleado como lenguaje principal, mientras que Jetpack Compose se ha utilizado para construir la interfaz nativa de la aplicación de forma declarativa. También se han usado los emuladores de Android Studio para realizar pruebas durante el desarrollo.

Para la comunicación entre Android y el backend se ha utilizado Retrofit, junto con OkHttp, permitiendo definir de forma estructurada las peticiones HTTP a la API REST. Para la carga de imágenes en Android se ha utilizado Coil, biblioteca optimizada para mostrar imágenes desde URLs dentro de interfaces Compose.

En el backend se han utilizado distintas bibliotecas y componentes. PHPMailer se ha empleado para el envío de correos electrónicos con códigos de verificación, recuperación de contraseña y cambios de correo. Altcha se ha utilizado como sistema CAPTCHA basado en proof-of-work para reducir registros o accesos automatizados. También se han usado PDO y sentencias preparadas para la comunicación segura con la base de datos.

Como repositorio del proyecto se ha utilizado GitHub, permitiendo mantener control de versiones, registrar cambios y conservar una copia remota del código fuente del TFG.

Para probar los endpoints de la API se ha utilizado Postman. Esta herramienta ha permitido enviar peticiones HTTP manuales, comprobar respuestas JSON, verificar autenticación con tokens y depurar errores del backend antes de integrar las funciones en la web o en Android.

En producción, la aplicación web y el backend se despliegan mediante Vercel. Para la base de datos y el almacenamiento de imágenes se utiliza Supabase. Supabase proporciona la base de datos relacional PostgreSQL y Supabase Storage, donde se guardan las imágenes de publicaciones y avatares de usuario.



7. IMPLEMENTACIÓN

7.1 ESTRATEGIA DE DESARROLLO

La implementación de IA-Lovers se llevó a cabo de forma incremental, partiendo primero del backend común y avanzando posteriormente hacia las dos interfaces cliente: la aplicación web y la aplicación Android. Esta estrategia permitió construir una base estable antes de desarrollar las funcionalidades visibles para el usuario.

En primer lugar, se implementó la API REST en PHP, ya que constituye el núcleo del sistema y concentra la lógica principal de la aplicación. A partir de esta API se definieron las rutas necesarias para gestionar usuarios, autenticación, publicaciones, comentarios, seguidores, notificaciones y etiquetas. Una vez comprobado el funcionamiento de cada endpoint, se integró progresivamente con la interfaz web.

La aplicación web se desarrolló como primera interfaz cliente, ya que permitía probar de forma rápida las funcionalidades del backend desde el navegador. Esto facilitó la detección temprana de errores en los flujos principales, como el registro, el inicio de sesión, la creación de publicaciones o la interacción con el feed.

Después se desarrolló la aplicación Android, reutilizando la misma API REST. De esta forma, la app móvil no necesitó una lógica de servidor independiente, sino que se conectó al backend ya existente mediante Retrofit. Esta decisión permitió mantener la coherencia de datos entre plataformas y comprobar que la arquitectura realmente funcionaba como sistema multiplataforma.

La implementación siguió una división por módulos funcionales. Cada bloque del sistema se desarrolló y validó de forma independiente antes de integrarlo con el resto. Por ejemplo, primero se implementó la autenticación, después la gestión de publicaciones, posteriormente los comentarios, los seguidores, las notificaciones y finalmente los ajustes de cuenta y detalles de integración.

7.2 CODIFICACIÓN Y DESARROLLO DE FUNCIONALIDADES

Durante la codificación se buscó mantener una separación clara entre las distintas responsabilidades del sistema. En el backend, los controladores reciben las peticiones HTTP, validan los datos principales y coordinan la respuesta. Los modelos se encargan del acceso a la base de datos, mientras que los servicios agrupan operaciones específicas como autenticación, envío de correos, almacenamiento de imágenes o validación CAPTCHA.

La autenticación se implementó mediante tokens Bearer. Cuando el usuario inicia sesión, el backend genera un token aleatorio, lo almacena asociado al usuario y lo devuelve al cliente. Tanto la web como Android guardan ese token y lo envían en las peticiones protegidas. Este mecanismo se utiliza en operaciones como publicar contenido, comentar, seguir usuarios, modificar el perfil o consultar notificaciones.

El registro de usuarios se desarrolló en varias fases para mejorar la seguridad. Primero, el usuario introduce sus datos y supera el CAPTCHA en la versión web. Después, el sistema crea un registro temporal y envía un código de verificación al correo electrónico. La cuenta solo se crea definitivamente cuando el usuario introduce el código correcto. Este mismo planteamiento se aplicó también a la recuperación de contraseña y al cambio de correo electrónico.

La funcionalidad de publicaciones se implementó mediante formularios multipart, permitiendo enviar imágenes junto con datos como título, descripción y etiquetas. En el backend se validan el tamaño, el tipo MIME real del archivo y los límites de texto antes de subir la imagen a Supabase Storage. La base de datos almacena únicamente la ruta del archivo y los metadatos necesarios, evitando guardar imágenes directamente en las tablas.

El feed de publicaciones se desarrolló con paginación basada en cursor. En la web, el scroll infinito se implementó mediante JavaScript, cargando nuevas publicaciones cuando el usuario se aproxima al final de la página. En Android, este comportamiento se adaptó al uso de listas en Jetpack Compose. Esta solución evita cargar todas las publicaciones de golpe y mejora el rendimiento de la plataforma.



Las interacciones sociales, como los “me gusta”, comentarios, respuestas y seguimientos, se implementaron como operaciones independientes en la API. Cada acción actualiza la base de datos y, cuando corresponde, genera una notificación para el usuario afectado. Por ejemplo, si un usuario comenta una publicación ajena, el sistema registra el comentario y crea una notificación para el autor de la publicación.

En la aplicación web, la lógica principal se concentró en archivos JavaScript como posts.js, auth.js, header.js y profile-settings.js. Estos archivos gestionan la comunicación con la API, el renderizado dinámico de publicaciones, el estado de sesión, el panel de notificaciones y los formularios de usuario.

En Android, la implementación se apoyó en ApiService.kt, ApiFactory.kt, modelos de datos Kotlin y pantallas desarrolladas con Jetpack Compose. El AppViewModel centraliza el estado principal de la aplicación y coordina las llamadas a la API. Esto permite que las pantallas reaccionen automáticamente cuando cambian los datos, por ejemplo al actualizar un feed, cambiar el estado de un “me gusta” o recibir nuevas notificaciones.

Los problemas encontrados durante el desarrollo se resolvieron mediante pruebas progresivas y revisión de cada flujo por separado. Cuando una funcionalidad no respondía correctamente, se comprobaba primero el endpoint con Postman, después la respuesta JSON del backend y finalmente la integración en web o Android.



7.3 INTEGRACIÓN DE LOS COMPONENTES

La integración de IA-Lovers se basa en una API REST común que actúa como punto de conexión entre todos los componentes del sistema. Tanto la web como la aplicación Android consumen los mismos endpoints, por lo que cualquier cambio realizado desde una plataforma queda reflejado en la otra.

El backend se integra con la base de datos mediante PDO y sentencias preparadas. Todas las operaciones importantes, como crear usuarios, consultar publicaciones, registrar comentarios o actualizar seguidores, pasan por esta capa de acceso a datos. Esto permite mantener centralizada la lógica de persistencia y evitar duplicar consultas en los clientes.

La integración con Supabase Storage se realiza desde el backend mediante la clase encargada del almacenamiento. Cuando un usuario sube una imagen, el cliente no se comunica directamente con Supabase, sino que envía el archivo al backend. Este valida el contenido, lo sube al almacenamiento externo y guarda la referencia en la base de datos. Posteriormente, cuando el feed solicita publicaciones, el backend devuelve URLs utilizables por la web y por Android.

La aplicación web se integra con la API utilizando peticiones fetch desde JavaScript. El token de autenticación se conserva en localStorage y se añade a las cabeceras de las peticiones protegidas. Además, la cabecera común de la web gestiona el estado de sesión y las notificaciones, de modo que el comportamiento sea coherente entre las distintas páginas.

En Android, la integración se realiza mediante Retrofit y OkHttp. Retrofit define los métodos correspondientes a cada endpoint, mientras que OkHttp gestiona la comunicación HTTP y permite añadir el token de autenticación a las peticiones. Las respuestas JSON se transforman en modelos Kotlin, lo que facilita trabajar con los datos dentro de las pantallas Compose.

La integración final del sistema se completa con el despliegue. La web y el backend se preparan para funcionar en Vercel, utilizando un wrapper en la carpeta api/ que redirige hacia el backend real. La base de datos y el almacenamiento de archivos se mantienen en Supabase.



8. PRUEBAS

8.1 Pruebas en la etapa de desarrollo

Durante el desarrollo de IA-Lovers se realizaron pruebas de forma continua, a medida que se iban implementando las distintas funcionalidades del sistema. El objetivo principal era comprobar cada parte por separado antes de integrarla con el resto de componentes.

En el backend, las pruebas se centraron en verificar el funcionamiento de los endpoints de la API REST. Para ello se utilizó Postman, enviando peticiones manuales a rutas como registro, login, creación de publicaciones, comentarios, seguidores, notificaciones y ajustes de usuario. Estas pruebas permitieron comprobar que las respuestas JSON tenían el formato esperado, que los códigos HTTP eran adecuados y que los errores se gestionaban correctamente.

También se probaron casos negativos, como intentos de inicio de sesión con credenciales incorrectas, códigos de verificación erróneos, archivos demasiado grandes, formatos de imagen no permitidos, peticiones sin token de autenticación o intentos de modificar recursos pertenecientes a otros usuarios.

Durante el desarrollo web se utilizó con frecuencia la consola del navegador mediante las herramientas de desarrollador, especialmente desde F12. La consola permitió detectar errores JavaScript, revisar mensajes de depuración y comprobar si las funciones se estaban ejecutando correctamente. Además, la pestaña Network resultó especialmente útil para analizar las peticiones HTTP enviadas al backend, revisar los datos enviados, comprobar las respuestas recibidas y detectar posibles fallos de comunicación entre cliente y servidor.

Estas herramientas también sirvieron para observar tiempos de carga, comprobar si las imágenes se descargaban correctamente desde Supabase Storage y detectar comportamientos inesperados en el feed, formularios o paneles dinámicos. Gracias a este tipo de pruebas fue posible corregir errores de forma temprana sin esperar a una fase final de validación.



Un ejemplo concreto de problema detectado durante estas pruebas apareció en la creación de publicaciones. Al pulsar el botón para publicar un post, la interfaz quedaba esperando la respuesta del backend. Si el usuario se impacientaba y volvía a pulsar el botón antes de recibir respuesta, se enviaban varias peticiones y podían crearse publicaciones duplicadas.

En la aplicación Android, las pruebas se realizaron tanto en emuladores de Android Studio como en un dispositivo físico Xiaomi. Se comprobó la navegación entre pantallas, el inicio de sesión, la carga de imágenes, la creación de publicaciones, la consulta de perfiles, las notificaciones y las interacciones sociales. El dispositivo físico permitió validar aspectos más cercanos al uso real, como la navegación táctil, el rendimiento de la interfaz y la visualización en una pantalla móvil.

La validación de integración consistió en comprobar que una acción realizada desde una plataforma se reflejaba correctamente en la otra. Por ejemplo, una publicación creada desde la web debía aparecer también en Android; un “me gusta” realizado desde Android debía actualizarse en el feed web; y los cambios de perfil debían mantenerse sincronizados al consultar de nuevo los datos desde cualquiera de los clientes.

Por último, se realizaron pruebas de despliegue para comprobar que el backend funcionaba correctamente fuera del entorno local, que las rutas públicas mantenían el formato esperado y que la conexión con Supabase permitía consultar datos y cargar imágenes correctamente. Estas pruebas permitieron asegurar que el prototipo no dependía únicamente del entorno de desarrollo local, sino que podía ejecutarse como una aplicación desplegada.



8.2 INTEGRACIÓN DE LOS COMPONENTES

Las pruebas de integración tuvieron como objetivo comprobar que los distintos componentes del sistema funcionaban correctamente entre sí. En IA-Lovers esto era especialmente importante porque el proyecto está formado por una aplicación web, una aplicación Android, un backend común, una base de datos y un sistema externo de almacenamiento de imágenes.

Una de las pruebas principales consistió en comprobar que una acción realizada desde una plataforma se reflejaba correctamente en la otra. Por ejemplo, una publicación creada desde la web debía aparecer también en Android; un “me gusta” realizado desde Android debía actualizarse al consultar el feed desde la web; y los cambios en el perfil del usuario debían mantenerse sincronizados en ambos clientes.

También se validó la integración entre el backend y Supabase Storage. Para ello se realizaron pruebas de subida de imágenes de publicaciones y avatares, comprobando que el archivo se almacenaba correctamente, que la ruta quedaba guardada en la base de datos y que posteriormente la imagen podía mostrarse tanto en la web como en Android.

Otra parte importante de la integración fue la autenticación. Se comprobó que los tokens generados por el backend pudieran ser utilizados correctamente por ambos clientes para acceder a rutas protegidas. También se verificó que, al cerrar sesión, el token dejara de ser válido y no pudiera seguir utilizándose para realizar operaciones privadas.

Las notificaciones también requirieron pruebas de integración, ya que dependen de acciones realizadas en otros módulos. Se comprobó que acciones como seguir a un usuario, dar “me gusta”, comentar una publicación o responder a un comentario generaran la notificación correspondiente para el usuario afectado.



8.3 PRUEBAS DE USUARIO, USABILIDAD Y FUNCIONALIDAD

Además de las pruebas realizadas durante el desarrollo, se pidió a varios amigos que probaran la versión web de IA-Lovers con el objetivo de detectar errores y evaluar la facilidad de uso de la plataforma. Estas pruebas fueron útiles porque permitieron observar cómo interactuaban usuarios externos con la aplicación, sin conocer previamente su funcionamiento interno.

Los usuarios probaron acciones habituales como registrarse, iniciar sesión, explorar publicaciones, crear posts, dar “me gusta”, comentar, seguir perfiles, consultar notificaciones y modificar datos de cuenta. A partir de estas pruebas se pudieron identificar problemas de usabilidad, comportamientos poco claros y pequeños fallos que durante el desarrollo podían pasar desapercibidos.

Este tipo de validación resultó especialmente importante en formularios y flujos de varios pasos, como el registro con código de verificación o la recuperación de contraseña. Al tratarse de procesos sensibles, era necesario comprobar que los mensajes fueran comprensibles, que el usuario supiera qué debía hacer en cada momento y que los errores se mostraran de forma clara.

También se revisó la experiencia general de navegación en la web, comprobando si el feed era cómodo de usar, si las publicaciones se entendían visualmente, si los botones principales eran fáciles de localizar y si la interfaz respondía correctamente en distintos tamaños de pantalla.



8.4 PRUEBAS DE DESEMPEÑO Y DE CARGA

Las pruebas de desempeño se centraron principalmente en comprobar que la aplicación respondiera de forma fluida al cargar publicaciones, imágenes y datos dinámicos. Y se evaluó el comportamiento del sistema con distintos volúmenes de publicaciones y mediante el uso continuado de la plataforma.

El feed se probó especialmente porque es una de las partes más importantes de una red social visual. Se comprobó que el scroll infinito cargara nuevas publicaciones progresivamente y que no fuera necesario descargar todo el contenido desde el inicio. Esta paginación basada en cursor ayuda a reducir el tamaño de las respuestas y mejora el rendimiento cuando aumenta el número de publicaciones.

Desde las herramientas de desarrollador del navegador se utilizó la pestaña Network para revisar los tiempos de respuesta de las peticiones, el tamaño de los recursos descargados y la carga de imágenes. Esto permitió comprobar si el backend respondía correctamente y si las imágenes alojadas en Supabase se servían de forma adecuada.

También se observaron aspectos de rendimiento en Android, especialmente en pantallas con listas de publicaciones e imágenes. El uso de Coil permitió cargar y cachear imágenes de forma más eficiente, evitando que la interfaz se bloqueara mientras se descargaban los recursos visuales.

No se ejecutaron pruebas intensivas de concurrencia. Aun así, se tuvieron en cuenta decisiones de diseño orientadas al rendimiento y a la escalabilidad, como el uso de Supabase Storage para imágenes, la paginación del feed y las consultas mediante sentencias preparadas en el backend.



8.5 PRUEBAS DE DESPLIEGUE

Las pruebas de despliegue se realizaron para comprobar que el sistema funcionaba correctamente fuera del entorno local. La web y el backend se desplegaron mediante Vercel, mientras que la base de datos y el almacenamiento de imágenes se gestionaron con Supabase.

Durante estas pruebas se comprobó que las rutas del backend mantuvieran el formato esperado, que el wrapper de la carpeta api/ redirigiera correctamente hacia el backend real y que las peticiones desde la web desplegada llegaran correctamente a la API. También se revisó que las variables de entorno estuvieran configuradas de forma adecuada para conectar con la base de datos, Supabase Storage, el sistema de correo y el CAPTCHA.

Vercel proporcionó herramientas útiles para esta fase, como los logs de ejecución, la información de deployments y las métricas básicas de Analytics. Los logs permitieron detectar errores producidos en producción, revisar respuestas fallidas y comprobar si una petición llegaba correctamente al backend. La información de despliegues permitió verificar qué versión estaba activa y revisar si los cambios se habían publicado correctamente.

Finalmente, se comprobó el funcionamiento general de los flujos principales en el entorno desplegado: registro, inicio de sesión, consulta del feed, creación de publicaciones, interacciones sociales, notificaciones y ajustes de cuenta. Con estas pruebas se verificó que el prototipo era utilizable no solo en desarrollo local, sino también en un entorno real de producción.



9. MANUALES

9.1 Manual de usuario

IA-Lovers permite explorar, publicar e interactuar con imágenes generadas mediante inteligencia artificial. Al entrar en la plataforma, el usuario puede visualizar publicaciones públicas desde el feed principal o desde la sección de exploración. Para realizar acciones como publicar, comentar, dar “me gusta”, seguir a otros usuarios o consultar notificaciones, es necesario iniciar sesión.

El usuario puede registrarse introduciendo un nombre de usuario, correo electrónico y contraseña. Después debe verificar su cuenta mediante un código enviado al correo. Una vez iniciada la sesión, puede crear publicaciones subiendo una imagen, añadiendo un título, una descripción opcional y etiquetas. Las publicaciones aparecerán en el feed general y en el perfil del usuario.

Desde cada publicación se puede acceder al detalle, dar “me gusta”, escribir comentarios o responder a otros comentarios. También se pueden visitar perfiles de otros usuarios, seguirlos y consultar sus publicaciones. En la sección de notificaciones se muestran avisos relacionados con nuevos seguidores, “me gusta”, comentarios o respuestas.

El usuario también dispone de una pantalla de ajustes donde puede modificar datos de cuenta, cambiar su avatar, actualizar correo o contraseña, cerrar sesión o eliminar su cuenta.



9.2 Manual de instalación

Versión web

La versión web no requiere instalación. El usuario solo necesita acceder a la URL de la plataforma desde un navegador compatible, registrarse o iniciar sesión y utilizar la aplicación directamente.

Versión Android

Para instalar la aplicación Android, se debe descargar la última versión del archivo APK desde el repositorio oficial:

<https://github.com/mikaelbry/IA-Lovers/releases/tag/apk>

La aplicación requiere como mínimo Android 8.0 Oreo, correspondiente al nivel de API 26. Una vez descargado el archivo .apk, el usuario debe abrirlo desde el dispositivo Android. Si el sistema lo solicita, será necesario permitir la instalación de aplicaciones de origen desconocido para el navegador o gestor de archivos utilizado.

Después se confirma la instalación y la aplicación quedará disponible en el dispositivo. Al abrir IA-Lovers, el usuario podrá iniciar sesión con una cuenta existente o registrarse desde la propia app.



10. POSIBLES TRABAJOS FUTUROS

10.1 Mejoras propuestas

Como trabajo futuro, IA-Lovers podría incorporar nuevas funcionalidades orientadas a mejorar la experiencia social y la gestión de la comunidad. Una mejora importante sería añadir un rol de moderador, con permisos específicos para revisar publicaciones, eliminar contenido inapropiado y gestionar usuarios que incumplan las normas de la plataforma.

También sería interesante implementar un sistema de mensajes directos entre usuarios. Esta función permitiría una comunicación más privada dentro de la red social, facilitando conversaciones entre creadores, intercambio de ideas o colaboración en proyectos relacionados con contenido generado por IA.

Otra mejora destacable sería integrar un generador de imágenes directamente dentro de la web y la aplicación Android. De esta forma, el usuario no tendría que crear las imágenes en herramientas externas antes de publicarlas, sino que podría generar contenido desde IA-Lovers y compartirlo inmediatamente en su perfil.

10.2 Expansión a nuevas plataformas

Como posible ampliación futura, se podría desarrollar una versión para iOS, permitiendo que usuarios de iPhone y iPad también pudieran acceder a la plataforma mediante una aplicación nativa.

10.3 Integración con nuevas tecnologías

IA-Lovers también podría evolucionar mediante la integración de nuevas tecnologías relacionadas con la inteligencia artificial generativa. Además de incorporar generación de imágenes, se podrían añadir funciones como recomendaciones personalizadas de publicaciones, clasificación automática por estilos o etiquetas, detección de contenido inapropiado y herramientas de ayuda para mejorar prompts.



11. CONCLUSIONES

11.1 Evaluación del proyecto

El desarrollo de IA-Lovers ha permitido alcanzar el objetivo principal planteado: crear una red social multiplataforma, disponible en web y Android, orientada a la publicación y consumo de imágenes generadas mediante inteligencia artificial.

También se han implementado las funcionalidades principales de una red social: registro e inicio de sesión, creación de publicaciones, feed general, búsqueda, “me gusta”, comentarios, respuestas, perfiles de usuario, seguimiento entre usuarios, notificaciones y configuración de cuenta. Además, se han aplicado medidas de seguridad como hash de contraseñas, tokens de autenticación, validación de archivos, CAPTCHA en la web, rate limiting y uso de sentencias preparadas.

11.2 Lecciones aprendidas

Durante el desarrollo he adquirido grandes conocimientos sobre la creación de aplicaciones multiplataforma conectadas a un backend común. Ya que hasta ahora nunca había hecho ningún proyecto de esta magnitud y había muchas palabras o funcionalidades que al empezar no entendía para nada. Y puedo decir que ya no es tan así.

También fue especialmente desafiante aprender a trabajar con Android, fueron tecnologías que supusieron una curva de aprendizaje bastante mayor respecto al desarrollo web. Pero también fue muy gratificante poder tener en un móvil una aplicación desarrollada 100% por mi.

Otro aspecto relevante fue comprender mejor la importancia de la validación y la seguridad. Funcionalidades aparentemente simples, como subir una imagen o cambiar un correo electrónico, requieren muchos controles adicionales para evitar errores, abusos o comportamientos no deseados.



12. OPINIÓN PERSONAL

Personalmente fue un proyecto que me supuso mucha mas carga de la que yo creí al principio. Pensé que en unos 2 meses ya estaría acabando todo pero apenas empezaba a darme cuenta de la magnitud de las cosas que tenía que tener en cuenta y que tenía que hacer. Pero por eso mismo creo que me ha hecho aprender mucho como desarrollador, en lo que me encontraré en el mercado laboral y entender en general mucho mejor como funcionan todas las aplicaciones que usamos en nuestro día a día.



13. GLOSARIO

A:

API (Application Programming Interface) — Interfaz de programación que permite la comunicación entre distintos componentes software.

API REST — Estilo de arquitectura para APIs que utiliza los verbos HTTP (GET, POST, PUT, DELETE) para operar sobre recursos.

APK (Android Package Kit) — Formato de empaquetado para distribuir e instalar aplicaciones en Android.

Altcha — Sistema CAPTCHA basado en proof-of-work que genera un reto matemático que el cliente debe resolver antes de enviar un formulario.

Android Studio — Entorno de desarrollo integrado (IDE) oficial para crear aplicaciones Android. Apache — Servidor web HTTP de código abierto muy utilizado en entornos PHP.

Avatar — Imagen o icono que representa a un usuario en una plataforma digital.

B:

Backend — Parte del sistema que se ejecuta en el servidor y gestiona la lógica de negocio, la base de datos y la autenticación.

Badge — Indicador visual que muestra un contador, normalmente sobre un icono, para informar de elementos pendientes (notificaciones no leídas).

bcrypt — Algoritmo de hash de contraseñas diseñado para ser lento y resistente frente a ataques de fuerza bruta.

Bearer token — Mecanismo de autenticación en HTTP donde el cliente incluye un token en la cabecera "Authorization: Bearer ".

Bucket — Contenedor lógico dentro de un sistema de almacenamiento de objetos (como Supabase Storage) donde se agrupan archivos.



Bundle — Paquete que contiene la aplicación Android compilada y sus recursos.

C:

CAPTCHA — Prueba automatizada para distinguir entre usuarios humanos y programas automatizados (bots).

CORS (Cross-Origin Resource Sharing) — Mecanismo de seguridad que permite o restringe peticiones HTTP desde dominios diferentes al del servidor.

CSS (Cascading Style Sheets) — Lenguaje utilizado para definir la presentación visual de documentos HTML.

Cache — Almacenamiento temporal de datos para acelerar accesos futuros. **Coil** — Biblioteca de carga de imágenes para Android basada en Kotlin que gestiona descarga, caché y visualización.

Composable — Función anotada con `@Composable` en Jetpack Compose que define un elemento de interfaz de usuario declarativa.

Compose (Jetpack Compose) — Toolkit moderno de Android para construir interfaces de usuario de forma declarativa usando código Kotlin.

Controlador (Controller) — Componente del backend que recibe peticiones HTTP, coordina la lógica y devuelve respuestas.

Corrutinas (Coroutines) — Mecanismo de concurrencia ligera en Kotlin que permite ejecutar código asíncrono sin bloquear el hilo principal.

Cursor — Marcador que indica la posición desde la cual continuar una paginación, evitando cargar datos ya recuperados.

D:

Deepfake — Contenido audiovisual sintético generado por IA que puede suplantar la apariencia o voz de una persona real.

Deployment — Proceso de poner una aplicación en un entorno de producción accesible para los usuarios finales.



DropDownMenu — Componente de interfaz que muestra un menú desplegable con opciones seleccionables.

E:

Endpoint — Dirección específica de una API que responde a una petición HTTP y realiza una función concreta.

Entorno de desarrollo — Configuración local donde se escribe y prueba el código antes de publicarlo.

Entorno de producción — Configuración real donde la aplicación está disponible para los usuarios finales.

F:

Feed — Flujo continuo de publicaciones que se muestran al usuario, normalmente ordenadas por relevancia o fecha.

fetch — API nativa de JavaScript para realizar peticiones HTTP asíncronas.

finfo — Extensión de PHP que permite determinar el tipo MIME real de un archivo analizando su contenido binario.

Flow token — Token temporal que identifica un proceso en curso (como un registro o cambio de correo) y permite continuarlo en pasos posteriores.

Frontend — Parte del sistema que ejecuta en el cliente (navegador o dispositivo) y con la que interactúa el usuario.

G:

GitHub — Plataforma de alojamiento de repositorios Git que permite control de versiones y colaboración en proyectos de software.

Gradle — Sistema de automatización de construcción utilizado en proyectos Android para compilar, empaquetar y gestionar dependencias.



H:

HTML (HyperText Markup Language) — Lenguaje de marcado utilizado para estructurar y presentar contenido en la web.

HTTP (Hypertext Transfer Protocol) — Protocolo de comunicación para la transferencia de información en la web. **HTTPS (HTTP Secure)** — Versión segura de HTTP que cifra la comunicación mediante TLS/SSL.

Hash — Función matemática unidireccional que transforma datos de entrada en una cadena de longitud fija (huella digital).

I:

IA (Inteligencia Artificial) — Rama de la informática que desarrolla sistemas capaces de realizar tareas que requieren inteligencia humana.

Interceptor — Componente de OkHttp que permite inspeccionar, modificar o agregar cabeceras a las peticiones HTTP antes de enviarlas.

IntersectionObserver — API de JavaScript que detecta cuándo un elemento entra o sale del área visible del navegador.

Inyección SQL (SQL Injection) — Tipo de ataque que inserta código SQL malicioso en las consultas a la base de datos.

J:

JSON (JavaScript Object Notation) — Formato ligero de intercambio de datos basado en texto, legible por humanos y fácil de procesar por máquinas.

JavaScript — Lenguaje de programación interpretado que se ejecuta en el navegador para dar interactividad a las páginas web.

Jetpack — Conjunto de bibliotecas oficiales de Android que facilitan el desarrollo siguiendo las mejores prácticas.

K:



Kanban — Metodología ágil de gestión de proyectos que organiza el trabajo en columnas visuales (pendiente, en progreso, completado).

Kotlin — Lenguaje de programación moderno para la JVM, oficialmente recomendado por Google para el desarrollo de aplicaciones Android.

L:

LazyColumn — Componente de Jetpack Compose que muestra una lista desplazable que solo renderiza los elementos visibles (carga perezosa).

Like (me gusta) — Acción que permite a un usuario mostrar aprecio por una publicación.

localStorage — Mecanismo de almacenamiento en el navegador que persiste datos entre sesiones sin fecha de expiración.

Log — Registro cronológico de eventos generados por una aplicación para facilitar la depuración y el monitoreo.

Login — Proceso de autenticación mediante el cual un usuario accede a un sistema con sus credenciales.

Logout — Acción de cerrar la sesión actual, invalidando el token de autenticación.

M:

MVC (Model-View-Controller) — Patrón de arquitectura que separa una aplicación en tres capas: datos (modelo), interfaz (vista) y lógica de control.

Material 3 (Material Design 3) — Sistema de diseño de Google que proporciona principios, componentes y estilos para interfaces modernas.

Media queries — Técnica de CSS que permite aplicar estilos según las características del dispositivo (ancho de pantalla, orientación, etc.).

Middleware — Capa de software que intercepta peticiones para realizar validaciones (como comprobar autenticación) antes de llegar al controlador.



Modelo — Componente que representa los datos y la lógica de acceso a la base de datos en una aplicación.

Multipart — Tipo de formulario HTTP que permite enviar datos binarios (como imágenes) junto con campos de texto.

N:

Navigation Compose — Biblioteca de Jetpack que permite gestionar la navegación entre pantallas mediante un grafo de rutas.

NavigationBar — Componente de Material 3 que muestra una barra de navegación inferior con iconos para cambiar entre secciones.

O:

OkHttp — Cliente HTTP eficiente para Java/Kotlin que gestiona conexiones, interceptores, caché y protocolos.

Optimistic update — Técnica que actualiza la interfaz inmediatamente antes de recibir la confirmación del servidor, revirtiendo si hay error.

P:

PDO (PHP Data Objects) — Extensión de PHP que proporciona una interfaz uniforme para acceder a distintas bases de datos con sentencias preparadas.

PHPMailer — Biblioteca PHP para el envío de correos electrónicos mediante SMTP con soporte de autenticación y adjuntos.

PHP — Lenguaje de script del lado del servidor ampliamente utilizado para desarrollo web y APIs REST.

Paginación — Técnica de dividir un conjunto grande de resultados en bloques más pequeños (páginas) para mejorar el rendimiento.



PostgreSQL — Sistema de base de datos relacional de código abierto con soporte avanzado de tipos de datos y consultas.

Postman — Herramienta gráfica para probar APIs que permite enviar peticiones HTTP y analizar respuestas.

Prepared statements — Consultas SQL parametrizadas que separan el código SQL de los datos, previniendo inyecciones SQL.

Prompt — Instrucción textual proporcionada a un modelo de IA generativa para producir una salida deseada.

Proof-of-work — Mecanismo de seguridad que requiere que el cliente realice un cálculo computacional antes de procesar una petición.

R:

REST (Representational State Transfer) — Conjunto de principios arquitectónicos para diseñar servicios web que utilizan los verbos HTTP estándar.

Rate limiting — Técnica de seguridad que limita el número de peticiones que un cliente puede realizar en un intervalo de tiempo.

Responsive — Diseño web que se adapta automáticamente al tamaño y orientación de la pantalla del dispositivo.

Retrofit — Biblioteca de tipado seguro para Android que convierte una interfaz Kotlin en peticiones HTTP a una API REST.

Router (enrutador) — Componente que analiza la URI de una petición y la dirige al controlador correspondiente.

S:

SQL (Structured Query Language) — Lenguaje estandarizado para gestionar y consultar bases de datos relacionales.

Scaffold — Componente de Material 3 que proporciona la estructura base de una pantalla (barra superior, inferior, contenido).



Scroll infinito — Técnica de carga progresiva que añade más contenido cuando el usuario se acerca al final de la página. Sentencias preparadas (Prepared statements) — Ver Prepared statements.

Singleton — Patrón de diseño que garantiza que una clase tenga una única instancia en toda la aplicación.

Splash screen — Pantalla inicial que se muestra mientras la aplicación verifica el estado de sesión o carga recursos iniciales.

Supabase — Plataforma backend como servicio (BaaS) que proporciona base de datos PostgreSQL, autenticación y almacenamiento.

Supabase Storage — Sistema de almacenamiento de objetos de Supabase para guardar y servir archivos (imágenes, avatares).

U:

URL (Uniform Resource Locator) — Dirección que identifica un recurso en internet.

URI (Uniform Resource Identifier) — Secuencia de caracteres que identifica de forma única un recurso.

V:

Vercel — Plataforma de despliegue en la nube especializada en aplicaciones web y funciones serverless.

ViewModel — Componente de la arquitectura Android que almacena y gestiona el estado de la interfaz sobreviviendo a cambios de configuración.

VSCodium — Editor de código fuente libre, derivado de VS Code sin telemetría.

X:

XAMPP — Paquete de software gratuito que incluye Apache, MySQL/MariaDB, PHP y Perl para entornos de desarrollo local.



IES LA ARBOLEDA
Avda. del Oeste, s/n
28922 - Alcorcón



UNIÓN EUROPEA
Fondo Social Europeo
El FSE invierte en tu futuro



14. BIBLIOGRAFÍA

1. Five Percent. (2025). More articles are now created by AI than humans. Graphite. <https://graphite.io/five-percent/more-articles-are-now-created-by-ai-than-humans#update>

2 MarketingNews. (2025, 26 de junio). El 65% de los usuarios ya se encuentra con contenido generado por inteligencia artificial en redes sociales, aunque muchos no lo detectan. <https://www.marketingnews.es/investigacion/noticia/1188853031605/el-65-de-los-usuarios-ya-se-encuentra-con-contenido-generado-por-inteligencia-artificial-en-redes-sociales-aunque-muchos-no-lo-detectan.1.html>

3 Congreso Nodos. (2024). Preferencia de los consumidores con respecto al contenido creado por inteligencia artificial generativa y el contenido creado por humanos [Ponencia]. <https://2024.nodos.org/ponencia/preferencia-de-los-consumidores-con-respecto-al-contenido-creado-por-inteligencia-artificial-generativa-y-el-contenido-creado-por-humanos/https://2024.nodos.org/ponencia/preferencia-de-los-consumidores-con-respecto-al-contenido-creado-por-inteligencia-artificial-generativa-y-el-contenido-creado-por-humanos/>

4 UNESCO. (2026, 11 de marzo). Inteligencia artificial y desinformación. <https://unes.co/z2cuga>

5 Bustos, D. (2025, 19 de agosto). Estado de la IA en profesionales de social media 2025 [Informe]. Metricool. <https://metricool.com/es/informe-uso-ia-redes-sociales/>